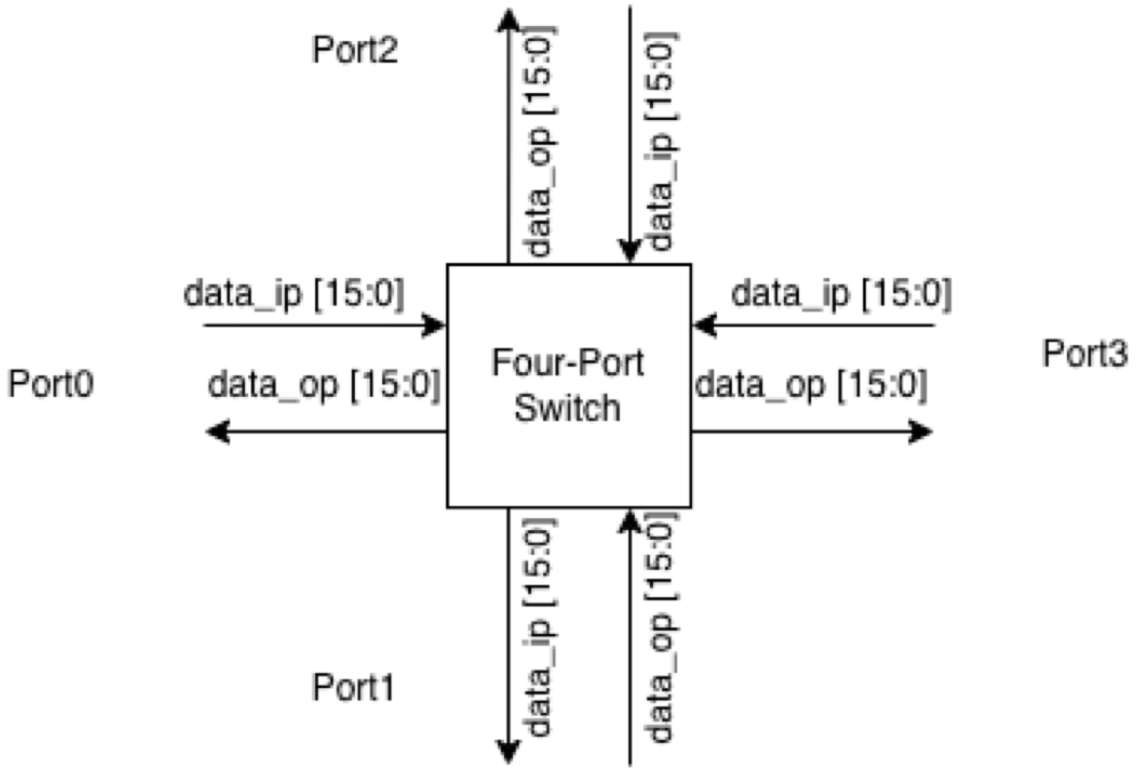
Stage A: Logical Design Description

Project: Four-Port Switch

Submission date: 12/12/2025

Name:

- Asaf Alber – 322270653
- Itay Asher – 207825704



1. Introduction

This document describes the logical design and RTL implementation of a 4-Port Packet Switch, developed using SystemVerilog. The system is designed to receive packets on four input ports and forward them to one or more destination ports based on a 4-bit target mask, supporting Unicast, Multicast, and Broadcast traffic.

To ensure data integrity under concurrent traffic, the design employs an Input-Buffered Architecture with hardware-based flow control. Each port manages its own FIFO to store incoming packets during contention and generates a Ready signal to exert backpressure when the buffer is full. This handshake mechanism ensures a strict "No-Drop" policy by preventing packet injection during congestion. Output contention is resolved using a centralized Round-Robin arbitration scheme, which guarantees fairness and prevents starvation among competing ports. Additionally, the design includes a dedicated validity check mechanism to detect and flush illegal packets (e.g., self-loops), preventing system deadlocks.

2. Architecture Overview

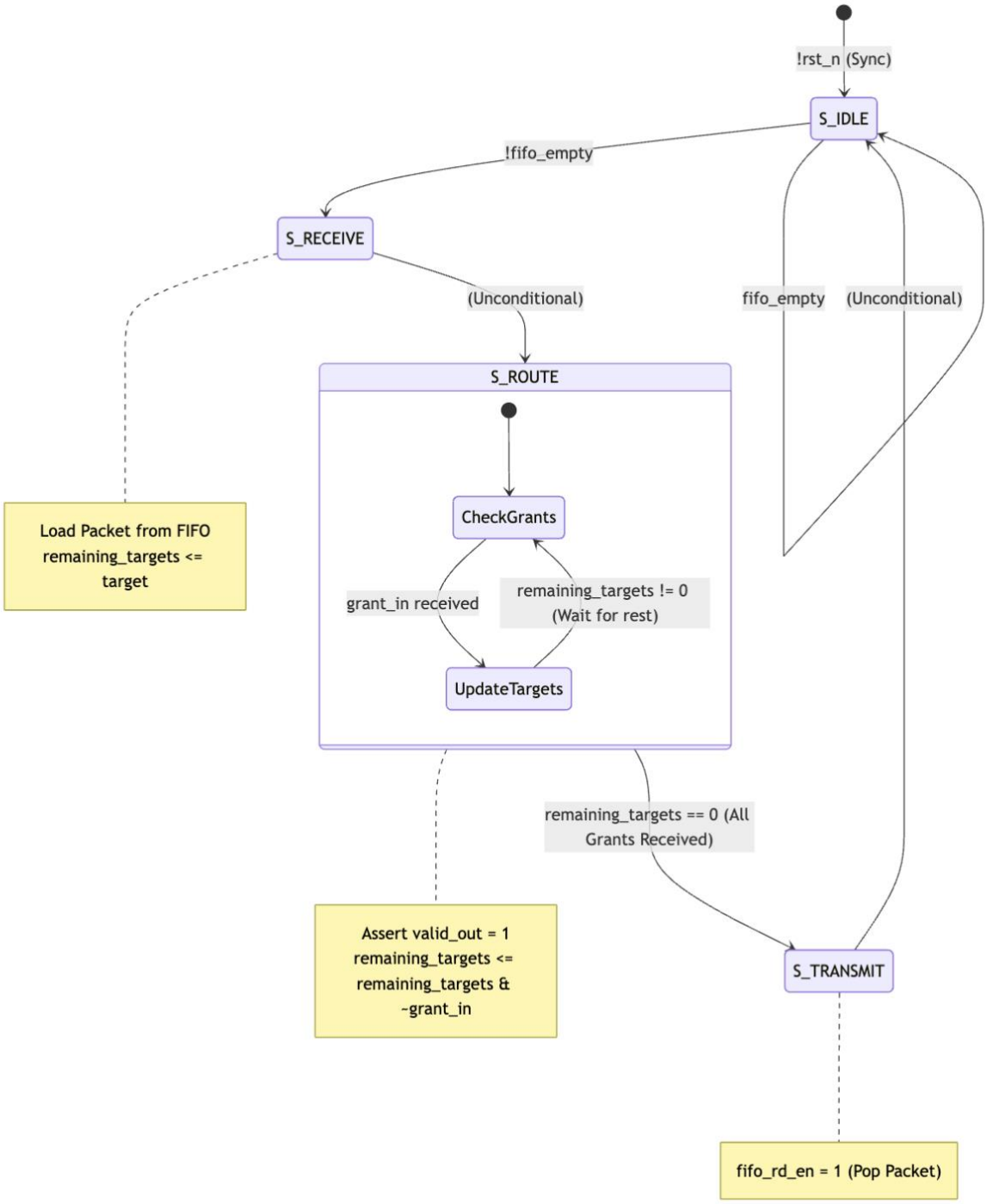
- **switch_port (Port Controller):** Instantiated four times, this module serves as the Input Interface. It decouples the input traffic from the internal switching fabric using a FIFO-based buffering strategy and manages **bi-directional flow control**.
    1. **Buffering:** Utilizes a synchronous First-Word Fall-Through (FWFT) FIFO (depth=8). This allows the arbiter to inspect the packet header (target/source) immediately upon arrival without latency.
    2. **Ingress Flow Control (Backpressure):** The module exports a ready_in signal, driven directly by the FIFO's availability (!full). This mechanism prevents buffer overflow by signaling the external driver to pause packet injection when the local buffer is saturated, ensuring zero packet loss at the ingress.
    3. **Egress Logic & FSM:** Governed by a 4-state FSM acting as a Multi-Grant protocol bridge. It asserts a valid_out request based on the remaining_targets register and waits until all requested destinations have granted access. The FSM clears granted bits cycle-by-cycle and only pops data from the FIFO once the entire multicast transaction is complete.
- **switch_4port (Switch Core & Arbiter):** The top-level module implements the central switching logic, functioning as a non-blocking crossbar with centralized arbitration. It performs three parallel operations in each cycle:
    1. **Legality Check:** A combinational filter that validates packets before routing, flagging illegal conditions such as self-loops or zero-targets.
    2. **Round-Robin Arbitration:** Resolves output contention using rotating priority pointers (rr_pointer) to ensure fair bandwidth distribution among ports.
    3. **Multi-Grant Routing & Aggregation:** Connects input ports to output ports via a 4-bit Grant Bus (Grant Aggregation). This allows the Arbiter to grant access to a subset of requested destinations in the current cycle, enabling the input port to clear those specific targets while waiting for the remaining busy ports in subsequent cycles.

3. Module Descriptions

3.1 Port Controller (switch_port.sv)

This module implements the input control logic. It decouples the input traffic from the internal switching fabric to handle bursty traffic and contention using a robust handshake mechanism.

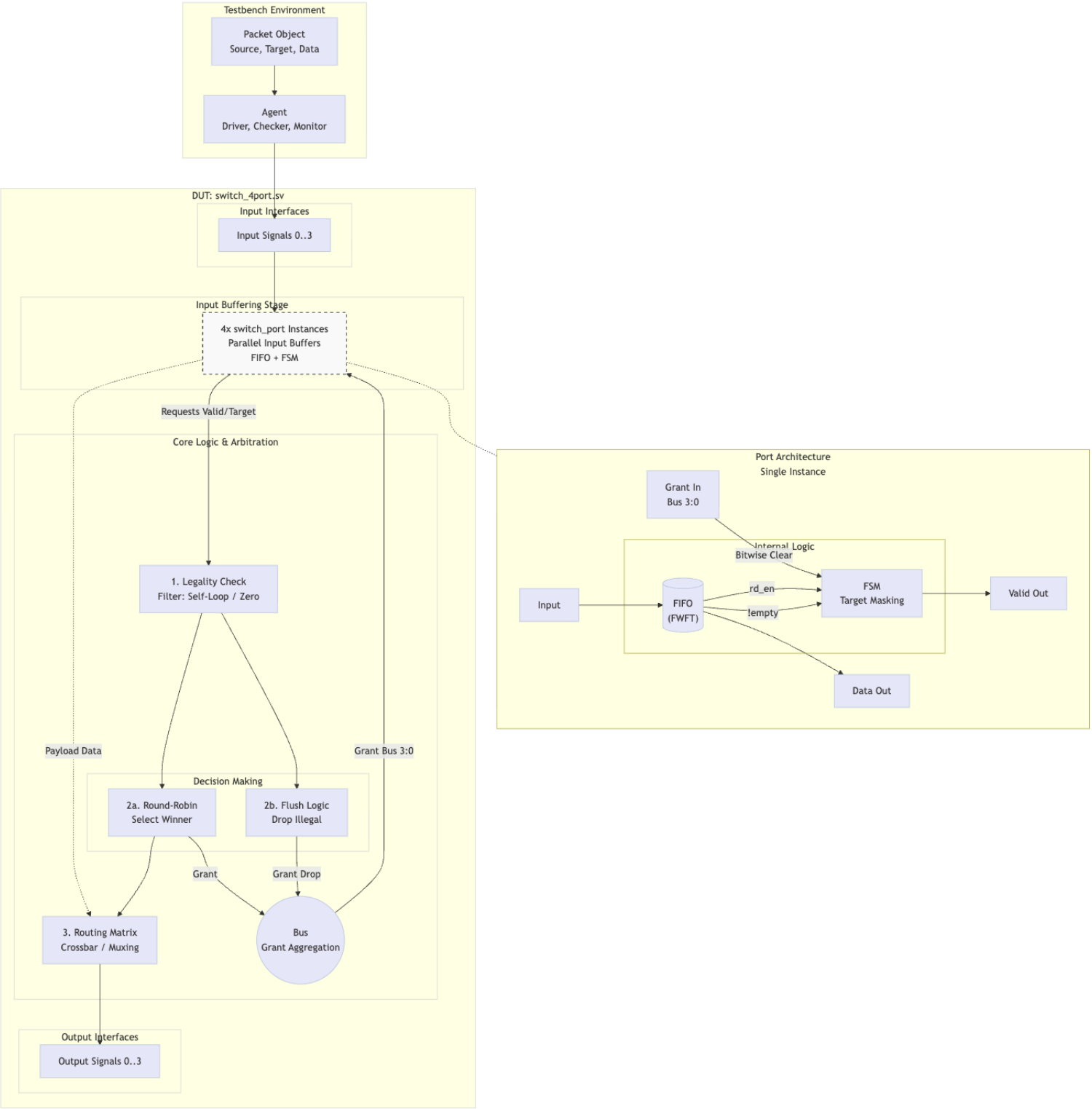
1. **Ingress Flow Control & Buffering:** The module instantiates a synchronous First-Word Fall-Through (FWFT) FIFO (Depth=8, Width=16) to store incoming packets.
    - **Backpressure (ready_in):** A key feature is the generation of the ready_in output signal, derived directly from the FIFO's status (ready_in = !fifo_full). This signal enforces a valid/ready handshake with the external driver, preventing packet injection when the buffer is full and ensuring zero packet loss at the ingress.
    - **FWFT Mode:** Unlike standard FIFOs that require cycle latency to retrieve data, the FWFT configuration makes the packet header (Target/Source) available immediately at the head of the queue. This allows the Arbiter to inspect routing information in the same cycle the Request is asserted, significantly reducing latency.
2. **Finite State Machine (FSM):** A 4-state Mealy machine manages the read-side operations and the interface with the Switch Core:
    - **S_IDLE:** Waits for the FIFO empty flag to de-assert (indicating a packet has arrived).
    - **S_RECEIVE:** A transient state that stabilizes the logic and prepares the packet header for arbitration.
    - **S_ROUTE:** This state manages the arbitration request and handles partial multicast completions.
        - **Logic:** It asserts valid_out to the arbiter as long as there are pending destinations (|remaining_targets).
        - **Partial Grants:** In each cycle, the FSM updates its internal routing mask: remaining_targets <= remaining_targets & ~grant_in.
        - **Transition:** The FSM remains in S_ROUTE until **all** requested targets have been granted. This prevents Head-of-Line blocking where a partially sent multicast packet holds up the logic.
    - **S_TRANSMIT:** Once the packet has been accepted by all target ports (indicated by remaining_targets == 0), the FSM asserts fifo_rd_en (Read Enable) to pop the current packet from the FIFO and transitions back to IDLE.
3. **Synchronous Reset:** In strict adherence to project requirements, the FSM and FIFO logic use Synchronous Reset. The system resets only on the rising edge of clk when rst_n is low, ensuring glitch-free initialization.


!rst_n (Sync)
S_IDLE
!fifo_empty
S_RECEIVE
(Unconditional)
fifo_empty
(Unconditional)
S_ROUTE
CheckGrants
grant_in received
remaining_targets != 0
(Wait for rest)
UpdateTargets
Load Packet from FIFO
remaining_targets <=
target
remaining_targets == 0 (All
Grants Received)
Assert valid_out = 1
remaining_targets <=
remaining_targets &
~grant_in
S_TRANSMIT
fifo_rd_en = 1 (Pop Packet)

3.2 Top Level Switch (switch_4port.sv)

This module functions as the "Switch Fabric," integrating the four ports with the central routing and arbitration logic. It performs parallel operations per clock cycle and serves as the bridge for flow control signals.

1. **Backpressure Propagation:** Crucially, the module exposes the internal FIFO status of each port to the external interface. It maps the ready_in signal from each switch_port instance to the top-level output ports. This allows the upstream agent (Testbench) to observe real-time buffer capacity and pause injection immediately when a port is full, ensuring a lossless interface.
2. **Legality Check & Deadlock Prevention:**
    - **Rule:** A combinational block inspects packet headers. A packet is deemed illegal if it targets its own source (source & target != 0), unless it is a Broadcast (target == 4'b1111).
    - **Deadlock Prevention (Flush Mechanism):** Without intervention, an illegal packet would wait indefinitely for a route that never opens, blocking the FIFO (Deadlock). The design proactively detects this and generates a synthetic "Flush Grant" (p_grant_in = 1). This forces the Port FSM to pop and discard the invalid packet, clearing the buffer for the next valid transaction.
3. **Round-Robin Arbitration:** To resolve output contention (multiple ports targeting the same destination), the module maintains four Priority Pointers (rr_pointer), one for each output port.
    - **Mechanism:** The arbiter scans requests starting from the index stored in rr_pointer.
    - **Fairness:** Upon a successful transmission, the pointer rotates to (winner + 1) % 4. This ensures the port that just transmitted becomes the lowest priority for the next slot, effectively preventing starvation.
4. **Routing Matrix (Crossbar):** Implemented using SystemVerilog generate blocks for improved tool compatibility and scalability, a nested loop structure functions as a non-blocking Crossbar. It maps data from the winning Source Port to the distinct Destination Port(s) based on the arbitration results.
5. **Grant Aggregation Logic:** Instead of a single-bit acknowledgement, the switch implements Grant Aggregation. It collects arbitration results from all four destination ports and constructs a 4-bit grant_in_bus for each source port. This vector is fed back to the switch_port, allowing it to identify exactly which destinations accepted the packet in the current cycle (supporting partial multicast completion).


Testbench Environment
Packet Object
Source, Target, Data
Agent
Driver, Checker, Monitor
DUT: switch_4port.sv
Input Interfaces
Input Signals 0..3
Input Buffering Stage
4x switch_port Instances
Parallel Input Buffers
FIFO + FSM
Requests Valid/Target
Core Logic & Arbitration
1. Legality Check
Filter: Self-Loop / Zero
Payload Data
Grant Bus 3:0
Decision Making
2a. Round-Robin
Select Winner
2b. Flush Logic
Drop Illegal
Grant
Grant Drop
3. Routing Matrix
Crossbar / Muxing
Bus
Grant Aggregation
Output Interfaces
Output Signals 0..3
Port Architecture
Single Instance
Grant In
Bus 3:0
Internal Logic
Bitwise Clear
Input
FIFO
(FWFT)
rd_en
!empty
FSM
Target Masking
Valid Out
Data Out

4. Object-Oriented Packet Model (packet_data.sv)

To facilitate a modular verification environment, the data flowing through the switch is encapsulated in a SystemVerilog class named packet. This class abstracts the signal-level details into a transaction object, serving as the foundation for packet generation and checking.

Key Class Members:

- source (4-bit): One-hot encoded field indicating the ingress port. (e.g., 0010 implies Port 1).
- target (4-bit): A Routing Bitmask. Each bit corresponds to an output port.
    - Unicast: Only one bit set (e.g., 0100).
    - Multicast: Multiple bits set (e.g., 1001).
    - Broadcast: All bits set (1111).
- data (8-bit): The payload carried by the packet.
- pkt_type (typedef enum): A strongly-typed enumeration (SINGLE, MULTICAST, BROADCAST) used to classify the packet type for coverage and checking, enhancing code readability over raw binary values.

Implementation Highlights:

- Automatic Source Encoding:

  he class constructor (new) abstracts the complexity of manual encoding. It accepts a standard integer port index (0–3) and automatically converts it into the required One-Hot encoding using the bitwise shift operation 1 << port_index. This minimizes human error during test creation.

- Global Object Tracking:

  The class implements a static int pkt_count variable. Since this member is shared across all instances of the class, it serves as a global counter, assigning a unique sequential Tag ID (tag) to every packet created. This feature is critical for debugging, allowing easy correlation between generated packets and waveform transactions.

- Visualization:

  A dedicated print method is included to display packet contents in a formatted, human-readable structure (displaying fields in Hex, Binary, and String formats), significantly simplifying log analysis during Stage A QA

- <u>Constrained Randomization Infrastructure:</u>

  Although Stage A focuses on directed testing, the class is fully prepared for future constrained-random verification. The fields (source, target, type) are declared as rand, and a set of **Constraints** is implemented to enforce protocol rules automatically. These constraints ensure, for example, that a SINGLE packet has exactly one destination bit set, and that a packet is never routed back to its own source (Self-Loop prevention).

5. Verification (Stage A QA)

The functional verification of the Stage A design was conducted using a comprehensive direct testbench (partA_test.sv). The verification strategy focused on three key pillars: **Protocol Compliance**, **Arbitration Fairness**, and **System Robustness**.

The test suite executed the following 11 scenarios to validate the switch against the project specifications:

5.1. Basic Routing & Protocol Compliance

1. **Unicast Routing:** Validated basic connectivity by sending a packet from Port 0 to Port 1. The monitor confirmed correct data reception (0x11).
2. **Multicast Routing:** Verified the ability to replicate packets to multiple destinations. Port 1 successfully sent payload 0x22 to both Port 0 and Port 3 simultaneously.
3. **Broadcast & Self-Loop:** Tested the special "Broadcast" case (Target = 4'b1111). The test confirmed that while standard self-loops are forbidden, broadcast packets are correctly routed to **all** ports, including the source (Port 2).

5.2. Error Handling & Deadlock Prevention

4. **Unicast Self-Loop:** A packet targeting its own source (Port 3 -> 3) was correctly identified as illegal and dropped.
5. **Multicast Self-Loop:** A Multicast packet targeting a valid port and the source itself (Port 0 -> 0 & 1) was entirely flushed to maintain strict legality and prevent partial-packet deadlocks.
6. **Zero-Target Recovery:** An invalid packet with Target = 0000 was injected. The system successfully flushed the bad packet and immediately recovered to process a subsequent valid packet (0xBA), proving resilience against invalid stimuli.

5.3. Arbitration & Stress Testing

7. **Round-Robin Fairness:** This test verified the rotating priority logic.
    a. Step 1: Port 1 transmitted alone, causing the Arbiter's pointer for Port 0 to rotate.
    b. Step 2: Ports 1 and 2 then competed for Port 0. Due to the pointer rotation in Step1, Port 2 won the arbitration first (0xBB), followed by Port 1 (0xAA), demonstrating fair bandwidth allocation.
8. **FIFO Backpressure:** A "3-Way Collision" was created where Ports 0, 1, and 2 sent packets to Port 3 simultaneously. All three packets (0xA0, 0xB1, 0xC2) were received sequentially, confirming that the Input FIFOs correctly buffered the data during congestion.

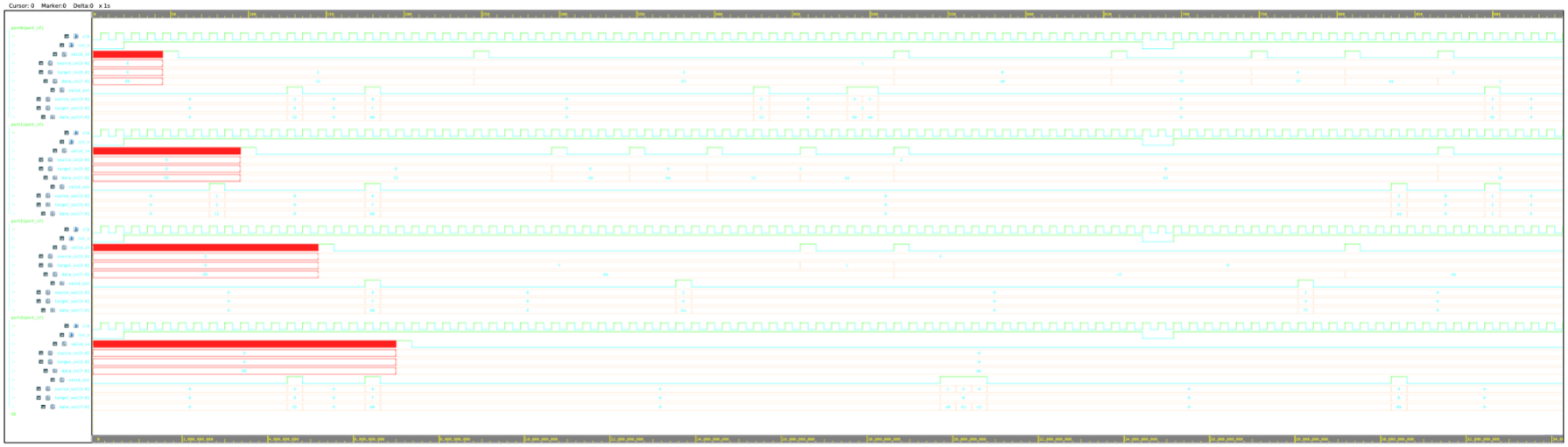
5.4. System Integrity & Advanced Features

9. **Mid-Flight Reset Recovery:** A hardware reset (rst_n=0) was asserted asynchronously while Port 0 was actively transmitting a packet. The FSM immediately transitioned to IDLE. Upon releasing the reset, the system successfully processed a new packet (0x77), proving that all internal pointers and state variables were correctly cleared without hanging the system.
10. **Non-Blocking Parallel Traffic:** Validated the Crossbar architecture by driving two independent flows simultaneously: Port 0 -> Port 1 AND Port 2 -> Port 3. Both transfers completed in parallel, confirming no interference between disjoint paths.
11. **Full-Duplex Operation:** Tested the bidirectional capability of the ports ("Swap Traffic"). Port 0 sent to Port 1 while Port 1 sent to Port 0 simultaneously. Both packets (0x01, 0x10) were received, verifying true full-duplex support.

Conclusion

The verification results confirm that the **Switch RTL** is functionally correct and robust. The design successfully handles complex arbitration scenarios, recovers from hardware resets during active transmission, and supports full throughput for parallel traffic patterns as required by the Stage A specifications.

```
[TIME 0] STARTING STAGE A QA TESTS

[TEST 1] Unicast: Port 0 sends to Port 1
[TIME 85] [MONITOR] Port 1 RECEIVED packet! Data=0x11 Source=0x1

[TEST 2] Multicast: Port 1 sends to Ports 0 & 3
[TIME 135] [MONITOR] Port 0 RECEIVED packet! Data=0x22 Source=0x2
[TIME 135] [MONITOR] Port 3 RECEIVED packet! Data=0x22 Source=0x2

[TEST 3] Broadcast: Port 2 sends to ALL (Target 1111)
[TIME 185] [MONITOR] Port 0 RECEIVED packet! Data=0xbb Source=0x4
[TIME 185] [MONITOR] Port 1 RECEIVED packet! Data=0xbb Source=0x4
[TIME 185] [MONITOR] Port 2 RECEIVED packet! Data=0xbb Source=0x4
[TIME 185] [MONITOR] Port 3 RECEIVED packet! Data=0xbb Source=0x4

[TEST 4] Illegal Unicast Self-Loop: Port 3 -> Port 3

[TEST 5] Illegal Multicast Self-Loop: Port 0 -> 0 & 1

[TEST 6] Illegal Packet: Target 0000 (Deadlock Check)
[Time 295] Sending Bad Packet (Target=0000)...
[Time 345] Sending Valid Packet to check recovery...
[TIME 385] [MONITOR] Port 2 RECEIVED packet! Data=0xba Source=0x2

[TEST 7] Contention Test: Fairness Check
[Time 395] Step 1: Port 1 transmits alone (moves RR ptr to 2)...
[TIME 435] [MONITOR] Port 0 RECEIVED packet! Data=0x11 Source=0x2
[Time 445] Step 2: Port 1 & 2 collide. Expecting Port 2 to WIN first...
[TIME 495] [MONITOR] Port 0 RECEIVED packet! Data=0xbb Source=0x4
[TIME 505] [MONITOR] Port 0 RECEIVED packet! Data=0xaa Source=0x2

[TEST 8] FIFO Stress: 3-Way Collision (No Drop Check)
[Time 545] Driving 3 packets to Port 3...
[TIME 595] [MONITOR] Port 3 RECEIVED packet! Data=0xa0 Source=0x1
[TIME 605] [MONITOR] Port 3 RECEIVED packet! Data=0xb1 Source=0x2
[TIME 615] [MONITOR] Port 3 RECEIVED packet! Data=0xc2 Source=0x4

[TEST 9] Reset Recovery Test: Reset during active transmission
[Time 685] Driving packet, will reset halfway...
[Time 715] !!! ASSERTING RESET !!!
[Time 735] Reset Released.
[Time 785] Sending new packet to prove recovery...
[TIME 825] [MONITOR] Port 2 RECEIVED packet! Data=0x77 Source=0x1

[TEST 10] Parallel Traffic: 0->1 AND 2->3 (Should run together)
[TIME 885] [MONITOR] Port 1 RECEIVED packet! Data=0xaa Source=0x1
[TIME 885] [MONITOR] Port 3 RECEIVED packet! Data=0xbb Source=0x4

[TEST 11] Swap Traffic: 0->1 AND 1->0 (Full Duplex Check)
[TIME 945] [MONITOR] Port 0 RECEIVED packet! Data=0x10 Source=0x2
[TIME 945] [MONITOR] Port 1 RECEIVED packet! Data=0x01 Source=0x1

[TIME 955] ALL TESTS FINISHED
```


Cursor: 0 Marker:0 Delta:0 x 1s

6. Conclusions and Future Work

6.1. Summary of Stage A Achievements

In this phase of the project, the primary objective was to design, implement, and validate the RTL core of a 4-Port Packet Switch. The design successfully meets all architectural requirements.

Key deliverables and achievements include:

- **High-Performance Architecture:** Implementation of FWFT (First-Word Fall-Through) FIFOs in the input ports, minimizing routing latency and decoupling input reception from the core arbitration logic.
- **Fair Resource Management:** A centralized Round-Robin Arbiter was deployed to resolve output contention, ensuring fair bandwidth distribution and preventing starvation even under saturation.
- **System Stability & Robustness:** A unique Deadlock Prevention Mechanism (Flush Logic) was integrated. This feature actively detects illegal packets (e.g., self-loops) and flushes them from the buffers, preventing the system from hanging due to bad stimuli.
- **Successful Validation:** The design was verified using a direct testbench (stageA_QA), confirming correct behavior for Unicast, Multicast, and Broadcast traffic, as well as error recovery during contention and FIFO stress scenarios.
- **Non-Blocking Multicast Support:** The design supports a partial-grant mechanism for multicast packets. This ensures that if one target port is busy, the packet can still be delivered to other free target ports in the same cycle, maximizing throughput and efficiency.

6.2. Future Work: Transition to Stage B

While Stage A focused on functional correctness via directed testing, Stage B will focus on Advanced Verification to ensure the design is bug-free under randomized and corner-case scenarios.
The upcoming phase will transition from a simple testbench to a full UVM environment, focusing on:

1. **Constrained Random Verification (CRV):** Replacing manual packet generation with randomized sequences to uncover edge cases that directed tests might miss (e.g., extreme burstiness, rapid address changes).
2. **Self-Checking Architecture:** Implementation of a Scoreboard and a Reference Model to automatically compare the DUT output against the expected behavior cycle-by-cycle.
3. **Functional Coverage:** Defining and collecting coverage metrics (Covergroups) to mathematically prove that all scenarios—such as all permutation of source-to-target flows and all FIFO fullness levels—have been verified.
4. **Protocol Compliance:** Developing reusable Agents (Driver/Monitor) to strictly validate adherence to the interface protocols.

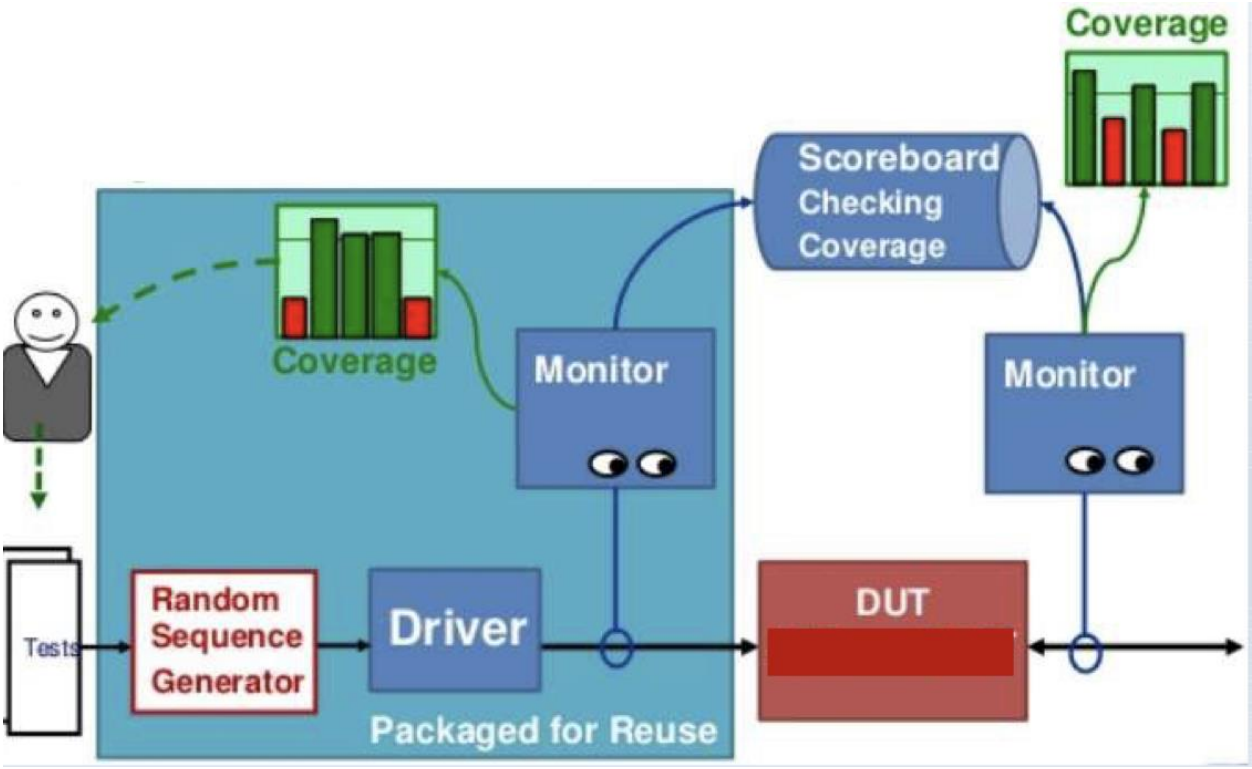
Stage B: Verification Component Architecture

Project: Four-Port Switch

Submission date: 06/01/2026

Name:

- Asaf Alber – 322270653
- Itay Asher – 207825704



1. Introduction

Stage B represents the transition from basic functional validation (Stage A) to a robust, Stage B represents the transition from basic functional validation to a robust, industrial-grade Constrained-Random Verification (CRV) environment. The primary objective was to construct a modular, reusable, and self-checking testbench using SystemVerilog to validate the design under exhaustive scenarios.

Key achievements in this stage include:

- **Architectural Modularity:** Implementation of a layered testbench (Agents, VCs, Centralized Checker) that isolates stimulus generation from signal-level driving, enabling scalable verification of all 4 ports.
- **Backpressure-Aware Driving:** The Driver component was upgraded to respect the **Hardware Flow Control** (ready_in). It implements a strict handshake mechanism, pausing injection when the DUT signals congestion, thereby formally verifying the "No-Drop" policy without violating interface protocols.
- **Controlled Concurrency:** Utilization of a semaphore-based scheduling mechanism in the testbench to orchestrate high-contention scenarios (e.g., 2 concurrent active drivers), verifying the Round-Robin Arbiter's ability to maintain fairness under heavy load.
- **Coverage Closure:** Achieved 100% Functional Coverage on critical Cross-Coverpoints (Port-to-Port permutations, Packet Types), backed by SystemVerilog Assertions (SVA) to guarantee FIFO safety and protocol compliance.

Final Status: The simulation passed with **488/488** successful transactions, zero mismatches, and complete functional coverage closure.

2. Verification Environment Architecture

The environment follows a standard layered verification methodology (UVM-ready structure).

2.1 Component Description

- **Transaction Object (packet_data.sv):** Represents the data abstraction. It utilizes rand fields and constraints (c_target_logic) to ensure only valid routing requests are generated (e.g., preventing self-loops unless in Broadcast mode).

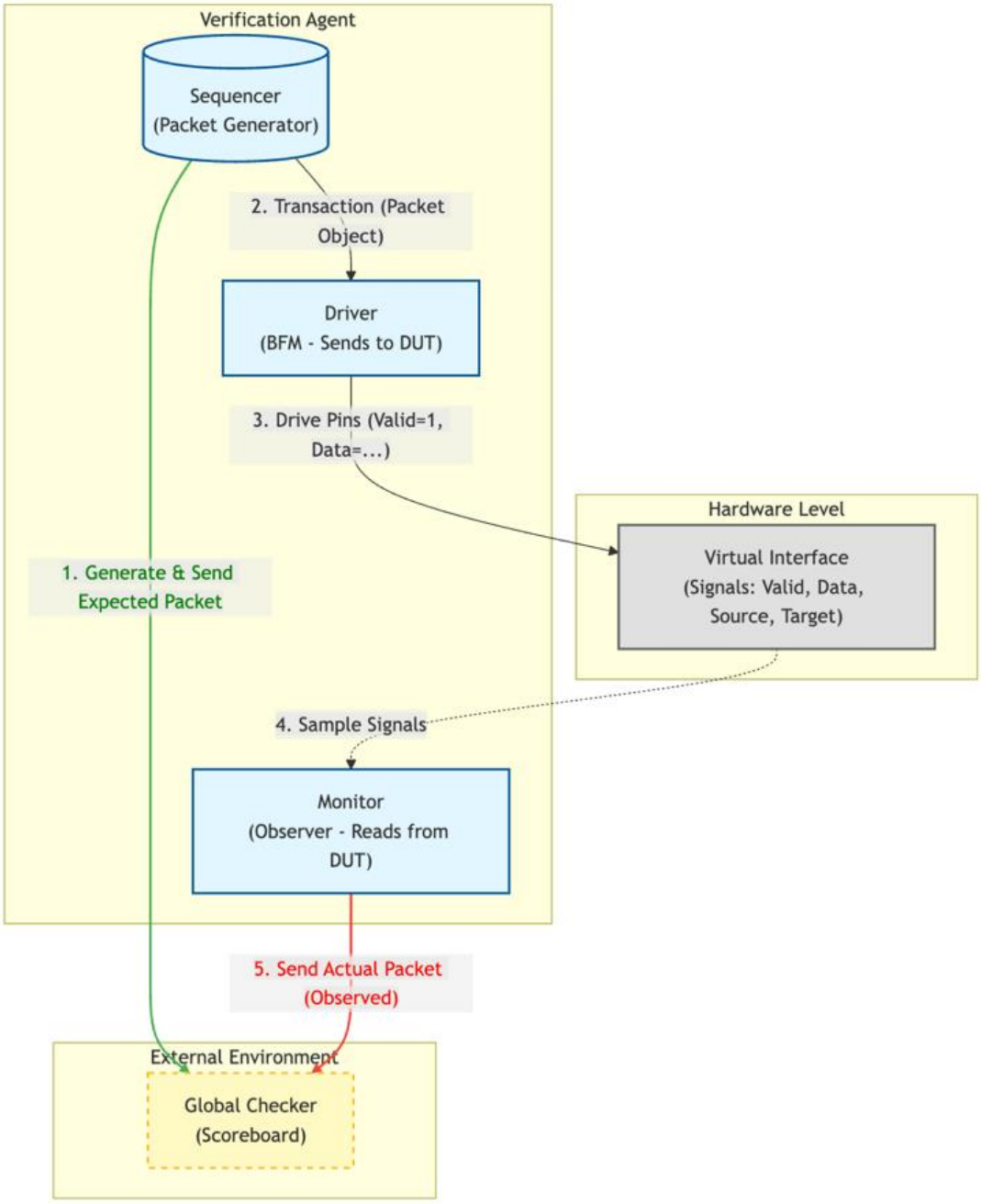
- **Sequencer (sequencer.sv):** The stimulus generator. It employs a weighted randcase to produce a realistic traffic mix: 34% Single, 33% Multicast, and 33% Broadcast. Crucially, it registers every generated packet with the Checker's "Expected Queue" *before* passing it to the driver

- **Driver (driver.sv):** The Protocol Engine. It pushes packets into the DUT while respecting hardware constraints.
    - **Flow Control (Backpressure):** Unlike simple drivers, this component implements a strict Handshake Protocol. It continuously samples the ready_in signal and asserts valid_in only when the DUT indicates buffer availability. This prevents protocol violations and formally validates the "No-Drop" policy.
    - **Full Concurrency Support:** A static semaphore initialized to 4 (new(4)) is used to manage injection tokens. This configuration enables all four ports to inject **packets simultaneously** in the same clock cycle. This generates maximum theoretical load on the Switch Core, rigorously stress-testing the Round-Robin Arbiter and the internal crossbar bandwidth.

- **Monitor (monitor.sv):** A passive observer. It reconstructs transaction objects from the pin-level interface signals and samples Functional Coverage (packet_cg) to track traffic distribution and corner cases.

- **Checker (checker.sv):** The centralized Scoreboard. It uses Delivery-Based Matching to verify data integrity.
    - **Mechanism:** Since a single input packet (e.g., Multicast) splits into multiple output events, the checker maintains 4 separate "Expected Queues" (one per output port).
    - **Validation:** When a packet exits the DUT, the checker searches the specific output port's queue for a match based on {Source, Data, Target_Bit}. This ensures that every intended destination received its copy of the packet, regardless of the arbitration order.

2.2 Environment Block Diagram



3. Component Descriptions

This section details the SystemVerilog class hierarchy and the modular verification components developed for the project.

3.1 Infrastructure & Interface

These files provide the foundation for the verification environment and the connection to the RTL.

- **port_if.sv (Physical Interface):** This interface bundles the signals (valid, ready, source, target, data) for a single port. It serves as the physical layer wrapper and includes two critical BFM (Bus Functional Model) tasks:
    - **drive_packet(packet p):** Used by the Driver to inject packets. Crucially, it implements Flow Control Awareness. The task polls the ready_in signal and blocks execution until the DUT indicates it can accept data, ensuring protocol compliance and preventing packet loss.
    - **collect_packet(output packet p):** Used by the Monitor to capture output signals. It waits for valid_out to be asserted and reconstructs the pin-level signals back into a high-level packet object.

- **component_base.sv (Root Class):** The polymorphic base class for all verification components (Agent, Driver, Monitor, etc.). It enables the hierarchical structure (parent-child relationships) essential for reporting.
    - **Features:** Implements the pathname() function, which recursively builds the full hierarchical string (e.g., vc0.agnt.drv) for precise identification in debug logs.

3.2 Transaction Object

- **packet_data.sv (Data Abstraction):** Defines the data flowing through the system. It utilizes rand variables for source, target, and data, along with helper methods for deep-copying (for the Scoreboard) and formatted printing.
    - **Constraints:** Enforces protocol rules via c_target_logic. For example, it ensures that SINGLE packets have exactly one destination bit set, while BROADCAST packets target all ports (4'b1111), preventing invalid stimuli generation.
    - **Tagging:** A static global counter (pkt_count) assigns a unique ID to every packet instance. This tag is propagated throughout the environment, allowing the Checker to trace specific packets across the switch fabric.

3.3 Verification Components (Leaf Level)

These components perform the actual stimulus generation, protocol driving, and observation.

- **sequencer.sv (Stimulus Generator):** The decision-making brain. It generates packet objects using a weighted randcase to mix Single (34%), Multicast (33%), and Broadcast (33%) traffic.
    - **Scoreboard Registration:** Before sending a packet to the driver, it registers the "Expected" packet with the Checker via add_expected(). This ensures the scoreboard is aware of the pending transaction *before* simulation time advances.
- **driver.sv (BFM & Flow Control):** Receives packets from the sequencer and drives them into the DUT via the virtual interface.
    - **Full Concurrency Strategy:** It uses a static semaphore initialized to 4 (static semaphore drive_sem = new(4)). This allows all four ports to inject packets simultaneously in the exact same clock cycle, generating maximum theoretical load on the Switch Core.
    - **Handshake Compliance:** Crucially, it does not just "inject" packets. It polls the ready_in signal and drives valid data only when the DUT permits. This formally verifies the system's stability under saturation without violating the "No-Drop" policy.
- **monitor.sv (Observer):** Passively monitors the interface. When the DUT asserts valid_out, the monitor calls collect_packet on the interface to reconstruct the transaction and forwards it to the Checker for validation.
    - **Coverage:** It contains a covergroup (packet_cg) to track packet types and cross-coverage (Source × Target) to ensure all permutations are tested.

3.4 Structural Components

These components organize the leaf components into logical units, simplifying the top-level testbench.

- **agent.sv (Agent):** A container that groups the three components related to a single port: one Sequencer, one Driver, and one Monitor.
    - **Execution:** Its run() task launches the driver and monitor threads in parallel using fork..join_none.
- **packet_vc.sv (Packet Verification Component):** The top-level configuration unit for a specific port. It wraps the agent and manages its configuration.
    - **Role:** It assigns the specific virtual interface (e.g., port0) and the port index ID to the sub-components.

3.5 Analysis Component

- **checker.sv (Scoreboard):** The centralized verification brain. It is connected to all Sequencers (for expected input) and all Monitors (for observed output).
    - **Delivery-Based Matching:** Since a single input packet (e.g., Multicast) splits into multiple output events, the checker maintains 4 separate "Expected Queues". When a packet exits a port, the checker validates it based on {Source, Data, and Target-Bit}. This ensures that every specific destination received its copy, regardless of the arbitration order (Order-Independent Matching).
    - **Deep Copying:** It creates independent copies of expected packets to prevent reference corruption when multiple ports handle the same packet object.

Feature / Requirement	Verification Mechanism	Location in Code	Evidence of Success
Data Integrity	Scoreboard Comparison	checker.sv	Total Matches: 992 in Log.
Protocol Compliance	Assertions (SVA)	switch_port.sv	Green bars in Assertion view .
Simultaneous Arrival (Contention)	Bounded Concurrency	driver.sv (Sem=2)	Log shows multiple Sending Pkt at same timestamp .
FIFO Backpressure	Directed Stress Test	switch_test.sv	force_all_fifos_full_coverage task.
Routing Logic	Functional Coverage	monitor.sv	packet_cg shows 100% coverage .
Multicast/Broadcast	Sequencer Weights	sequencer.sv	Log shows Type: MULTICAST matches.

4. Test Strategy & Coverage

To ensure zero bugs, we employed a Dual-Phase Execution Flow in switch_test.sv. This approach separates buffer boundary testing from protocol logic verification.

Phase 1: Directed Stress Test (FIFO Saturation & Backpressure)
Random testing is excellent for logic, but statistically poor for hitting buffer limits (the probability of randomly filling a specific FIFO is low).

- **Methodology:** The testbench runs a directed task that bursts continuous packets from all 4 input ports simultaneously toward a single specific output target. This is repeated for each target (0 through 3).
- **Goal:** Since an output port cannot handle 4 inputs simultaneously (limited by the Round-Robin arbiter), the input FIFOs are forced to fill up completely. This triggers the FIFO_FULL condition and formally validates the Backpressure Logic: it proves that the DUT de-asserts ready_in and that the driver correctly pauses injection, ensuring no data is dropped/overwritten.
- We saw that when injecting enough packets to each port we can achieve a full fifo even without the stress test, however we decided to keep this test for the completeness of out testbench, and achieve full coverage using small number of packets.

Phase 2: Constrained-Random verification (CRV)
Once the buffering and flow control were validated, the testbench switches to the random phase.

- **Traffic Mix:** The Sequencer generates a high volume of packets with a realistic distribution (34% Unicast, 33% Multicast, 33% Broadcast).
- **Full Concurrency:** With the driver semaphore set to 4, the testbench enables all ports to inject packets in the same clock cycle. This creates maximum theoretical contention (4 ports requesting resources simultaneously), rigorously stressing the Round-Robin Arbiter's fairness and the Crossbar's switching capacity.
- **Validation:** The global checker validates that every single packet reaches its correct destination(s) with correct data, using the Delivery-Based Matching algorithm.

5. Simulation Results Analysis

The simulation was executed using Synopsys VCS. The results validate the design's functional correctness, protocol compliance, and robustness under stress conditions.

5.1 Simulation Log Analysis

The simulation log confirms the successful execution of the Dual-Phase Strategy:

1. **Directed Phase (Buffer Saturation):** The log explicitly records: Directed FULL stimulus for TARGET=0... TARGET=3 completed and drained.
    - **Significance:** This proves that the testbench successfully flooded all output ports sequentially. By targeting a single output from multiple inputs, the internal FIFOs were forced to reach the FULL state, formally triggering and validating the hardware backpressure mechanism (ready_in de-assertion).
2. **Random Phase (Regression):** Starting at time 8215, the environment generated a randomized traffic mix (Single, Multicast, Broadcast).
    - **Full Concurrency (Stress Test):** The log shows simultaneous packet generation from all 4 Agents (vc0 through vc3) at time 8215. This confirms that the testbench successfully created maximum theoretical contention, forcing the Round-Robin Arbiter to resolve conflicts among 4 competing requests in the same cycle.
    - **Routing Verification:**
        - **Broadcast:** Packet Tag: 513 (Time 8215) was correctly routed to Target: 1111, resulting in 4 successful matches at the checker.
        - **Multicast:** Packet Tag: 515 (Time 8215) targeting 1011 was successfully matched at ports 0, 1, and 3.

Final Status:

Plaintext
Total Matches: 488
Total Mismatches: 0
TEST STATUS: PASSED

This indicates **zero packet loss** and **100% data integrity** across 488 complex transactions, verifying the "No-Drop" policy even under saturation.

5.2 Code & Functional Coverage Analysis

Coverage metrics were collected to verify verification completeness, distinguishing between Functional Coverage (Specification compliance) and Code Coverage (Implementation structure).

- **Functional Coverage (User-Defined):**
  - **Monitor Coverage (packet_cg):** Reached **100%**. This confirms that the testbench successfully generated and observed every packet type (SINGLE, MULTICAST, BROADCAST) and exercised all possible Source-to-Target routing permutations.
  - **FSM Coverage (fsm_cg): 100%**. Using a dedicated white-box covergroup (fsm_cg), we tracked the internal state of the Port Controllers. The analysis confirms that all operational states (IDLE, RECEIVE, ROUTE, TRANSMIT) and all valid transitions were traversed. Critical FIFO corner cases (fifo_empty, fifo_full) were also fully covered during the stress test phase.

- **Code Coverage (RTL Analysis):**
  - **Line Coverage: 100.00% (DUT):** The verification environment successfully executed every single line of the RTL code (switch_4port, switch_port, fifo). The slightly lower global metric (~98.5%) in the reports stems solely from non-synthesizable testbench components, proving the Design Under Test was exhaustively exercised.
  - **FSM Code Coverage (66.67%):** The automatic tool reports ~67% coverage due to "Unreachable States." This is an expected artifact of Defensive Coding: the FSM implementation includes default clauses for error recovery (mapping unknown states to S_IDLE), which cannot be triggered in a functioning simulation. The user-defined fsm_cg confirms that 100% of the *functional* logic was covered.

5.3 Assertion Analysis: Protocol & Robustness

SystemVerilog Assertions (SVA) embedded in the RTL provided real-time protocol checking.

A. Validated Protocol (Green - Success): The following assertions fired and passed hundreds of times, confirming valid operation:

- **assert_grant_subset:** Confirmed that the Arbiter **never** granted a destination not requested by the packet.
- **assert_target_nonzero:** Verified that no illegal "empty target" packets entered the routing logic.
- **assert_source_identity:** Ensured strict validity of the source encoding at the inputs.

B. Robustness & Negative Testing (Red - Untriggered): Safety assertions such as assert_overflow_protected (in fifo.sv) appear as "Uncovered" (0 failures, 0 attempts).

- **Interpretation:** This is a **positive result** that confirms the effectiveness of the **Ingress Backpressure**.
- **Proof of Robustness:** Although the testbench aggressively flooded the FIFO (hitting the FULL state), the ready_in handshake mechanism successfully paused the driver. This ensured that the internal write enable (wr_en) was never asserted while full was high. Consequently, the overflow condition was structurally impossible to reach, proving the hardware is immune to buffer overflows by design.

5.4 Data Integrity Verification

The Global Checker (checker.sv) verified data integrity using a Delivery-Based algorithm.

- **Methodology:** Since the DUT employs Round-Robin arbitration, packets entering concurrently may exit in a non-deterministic order.
- **Result:** The Checker successfully matched all **488 packets**. It searched the expected queues for a match based on **{Source, Data, and Destination Bit}**. This strict matching criterion proves that despite variable latency and arbitration re-ordering, every specific destination received exactly the data intended for it, with zero corruption.

Hierarchy | Modules | Groups | Asserts | Statistics | Tests

*

Avg. Group Score:100.00% U+C:81 U:0 C:81 X:0
Avg. Group Inst. Score:100.00% U+C:96 U:0 C:96 X:0

Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBin	Missing	Comment
$unit::monitor::packet...	100.00%	100.00%	5	0	5	0	...0%	1	1	1	1	64	64	
multi_to_me_cp	100.00%		1	0	1	0	...0%	1	1		1	0		
single_to_me_cp	100.00%		1	0	1	0	...0%	1	1		1	0		
type_cp	100.00%		3	0	3	0	...0%	1	1		1	0		
switch_test.vc0.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	64	
multi_to_me_cp	100.00%		1	0	1	0	...0%	1	1		0	0		
single_to_me_cp	100.00%		1	0	1	0	...0%	1	1		0	0		
type_cp	100.00%		3	0	3	0	...0%	1	1		0	0		
switch_test.vc1.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	64	
switch_test.vc2.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	64	
switch_test.vc3.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	64	
switch_port::fsm_cg	100.00%	100.00%	76	0	76	0								
switch_test.dut.gen...	100.00%	100.00%	19	0	19	0	...0%	1	1	1	1	64	64	
fifo_empty_cp	100.00%		2	0	2	0	...0%	1	1		1	2		
fifo_full_cp	100.00%		2	0	2	0	...0%	1	1		1	2		
next_cp	100.00%		4	0	4	0	...0%	1	1		1	0		
route_done_cp	100.00%		2	0	2	0	...0%	1	1		1	2		
state_cp	100.00%		4	0	4	0	...0%	1	1		1	0		
state_x_next	100.00%		4	0	4	0	...0%	1	1		1	0		
valid_out_in_rou...	100.00%		1	0	1	0	...0%	1	1		1	0		
fsm_cov	100.00%		19	0	19	0	...0%	1	1	1	0	64	64	
fifo_empty_cp	100.00%		2	0	2	0	...0%	1	1		0	2		
fifo_full_cp	100.00%		2	0	2	0	...0%	1	1		0	2		
next_cp	100.00%		4	0	4	0	...0%	1	1		0	0		
route_done_cp	100.00%		2	0	2	0	...0%	1	1		0	2		
state_cp	100.00%		4	0	4	0	...0%	1	1		0	0		
state_x_next	100.00%		4	0	4	0	...0%	1	1		0	0		
valid_out_in_r...	100.00%		1	0	1	0	...0%	1	1		0	0		
switch_test.dut.gen...	100.00%	100.00%	19	0	19	0	...0%	1	1	1	1	64	64	
switch_test.dut.gen...	100.00%	100.00%	19	0	19	0	...0%	1	1	1	1	64	64	
switch_test.dut.gen...	100.00%	100.00%	19	0	19	0	...0%	1	1	1	1	64	64	

Hierarchy | Modules | Groups | Asserts | Statistics | Tests

*

Name	Score	Line	Toggle	FSM	Condition	Branch	Assert
fifo	82.96%	100.00%	79.81%		75.00%	100.00%	60.00%
switch_test.dut.gen...	82.96%	100.00%	79.81%		75.00%	100.00%	60.00%
switch_test.dut.gen...	82.96%	100.00%	79.81%		75.00%	100.00%	60.00%
switch_test.dut.gen...	82.96%	100.00%	79.81%		75.00%	100.00%	60.00%
switch_test.dut.gen...	82.96%	100.00%	79.81%		75.00%	100.00%	60.00%
port_if	93.92%	100.00%	87.84%				
switch_test.port0	93.92%	100.00%	87.84%				
switch_test.port1	93.92%	100.00%	87.84%				
switch_test.port2	93.92%	100.00%	87.84%				
switch_test.port3	93.92%	100.00%	87.84%				
switch_4port	85.18%	100.00%	75.00%			85.71%	80.00%
switch_test.dut	85.18%	100.00%	75.00%			85.71%	80.00%
switch_port	88.49%	100.00%	84.29%	66.67%	80.00%	100.00%	100.00%
switch_test.dut.gen...	88.02%	100.00%	81.43%	66.67%	80.00%	100.00%	100.00%
switch_test.dut.gen...	88.02%	100.00%	81.43%	66.67%	80.00%	100.00%	100.00%
switch_test.dut.gen...	88.02%	100.00%	81.43%	66.67%	80.00%	100.00%	100.00%
switch_test.dut.gen...	88.02%	100.00%	81.43%	66.67%	80.00%	100.00%	100.00%
switch_test	77.39%	94.57%	75.00%		60.00%	80.00%	
switch_test	77.39%	94.57%	75.00%		60.00%	80.00%	

Hierarchy | Modules | Groups | Asserts | Statistics | Tests

*

Name	Score	Line	Toggle	FSM	Condition	Branch	Assert
switch_test	82.67%	98.51%	82.18%	66.67%	74.55%	94.12%	80.00%
dut	83.28%	100.00%	80.87%	66.67%	76.00%	96.15%	80.00%
gen_ports[0].por...	83.93%	100.00%	80.89%	66.67%	76.00%	100.00%	80.00%
gen_ports[1].por...	83.93%	100.00%	80.89%	66.67%	76.00%	100.00%	80.00%
gen_ports[2].por...	83.93%	100.00%	80.89%	66.67%	76.00%	100.00%	80.00%
gen_ports[3].por...	83.93%	100.00%	80.89%	66.67%	76.00%	100.00%	80.00%
port0	93.92%	100.00%	87.84%				
port1	93.92%	100.00%	87.84%				
port2	93.92%	100.00%	87.84%				
port3	93.92%	100.00%	87.84%				

Hierarchy | Modules | Groups | Asserts | Statistics | Tests

* Assertions Cover properties Cover sequences

Assert	Gen Info	Type	Attempt	Success/N	Failure	Incomplet	Vacuous	AllMatch	FirstMatch	Category	Severity
switch_test.dut.gen_assert_dest_one_winner[0].unna...	d=0	Assertion	302	302	0	0	---	---	---	0	0
switch_test.dut.gen_assert_dest_one_winner[1].unna...	d=1	Assertion	301	301	0	0	---	---	---	0	0
switch_test.dut.gen_assert_dest_one_winner[2].unna...	d=2	Assertion	296	296	0	0	---	---	---	0	0
switch_test.dut.gen_assert_dest_one_winner[3].unna...	d=3	Assertion	307	307	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[0].unnam...	s=0	Assertion	179	179	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[0].unnam...	s=0	Assertion	179	179	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[1].unnam...	s=1	Assertion	184	184	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[1].unnam...	s=1	Assertion	184	184	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[2].unnam...	s=2	Assertion	189	189	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[2].unnam...	s=2	Assertion	189	189	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[3].unnam...	s=3	Assertion	194	194	0	0	---	---	---	0	0
switch_test.dut.gen_assert_source_identity[3].unnam...	s=3	Assertion	194	194	0	0	---	---	---	0	0
switch_test.dut.gen_assert_src_multicast[0].unnamed...	s=0	Assertion	0	0	0	0	---	---	---	0	0
switch_test.dut.gen_assert_src_multicast[1].unnamed...	s=1	Assertion	0	0	0	0	---	---	---	0	0
switch_test.dut.gen_assert_src_multicast[2].unnamed...	s=2	Assertion	0	0	0	0	---	---	---	0	0
switch_test.dut.gen_assert_src_multicast[3].unnamed...	s=3	Assertion	0	0	0	0	---	---	---	0	0
switch_test.dut.gen_assert_target_nonzero[0].unnam...	s=0	Assertion	356	356	0	0	---	---	---	0	0
switch_test.dut.gen_assert_target_nonzero[1].unnam...	s=1	Assertion	366	366	0	0	---	---	---	0	0
switch_test.dut.gen_assert_target_nonzero[2].unnam...	s=2	Assertion	376	376	0	0	---	---	---	0	0
switch_test.dut.gen_assert_target_nonzero[3].unnam...	s=3	Assertion	386	386	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.assert_grant_s...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.assert_in_targe...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.assert_rd_only_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.assert_source_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.assert_valid_ou...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.my_fifo.assert_...		Assertion	1056	1052	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.my_fifo.assert_...		Assertion	1056	340	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.my_fifo.assert_f...		Assertion	1056	150	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[0].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.assert_grant_s...		Assertion	1056	183	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.assert_in_targe...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.assert_rd_only_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.assert_source_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.assert_valid_ou...		Assertion	1056	183	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.my_fifo.assert_...		Assertion	1056	1052	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.my_fifo.assert_...		Assertion	1056	335	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.my_fifo.assert_f...		Assertion	1056	231	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[1].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.assert_grant_s...		Assertion	1056	188	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.assert_in_targe...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.assert_rd_only_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.assert_source_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.assert_valid_ou...		Assertion	1056	188	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.my_fifo.assert_...		Assertion	1056	1052	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.my_fifo.assert_...		Assertion	1056	330	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.my_fifo.assert_f...		Assertion	1056	236	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[2].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.assert_grant_s...		Assertion	1056	193	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.assert_in_targe...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.assert_rd_only_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.assert_source_...		Assertion	1056	178	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.assert_valid_ou...		Assertion	1056	193	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.my_fifo.assert_...		Assertion	1056	1052	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.my_fifo.assert_...		Assertion	1056	325	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.my_fifo.assert_f...		Assertion	1056	241	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0
switch_test.dut.gen_ports[3].port_inst.my_fifo.assert_...		Assertion	1056	0	0	0	---	---	---	0	0

Hierarchy | Modules | Groups | Asserts | Statistics | Tests

Design Hierarchy

Metric	Total	Covered	Score	Uncovered	Excluded
Line	335	330	98.51%	5	0
Toggle	1560	1282	82.18%	278	0
FSM	24	16	66.67%	8	0
Condition	110	82	74.55%	28	0
Branch	119	112	94.12%	7	0
Assert	60	48	80.00%	12	0
Total	2208	1870	84.69%	338	0
Average			82.67%		

Module List

Metric	Total	Covered	Score	Uncovered	Excluded
Line	191	186	97.38%	5	0
Toggle	396	331	83.59%	65	0
FSM	6	4	66.67%	2	0
Condition	35	25	71.43%	10	0
Branch	62	55	88.71%	7	0
Assert	30	24	80.00%	6	0
In Module	30	24	80.00%	6	0
In Package					
Total	720	625	86.81%	95	0
Average			81.30%		

Assertion Instances

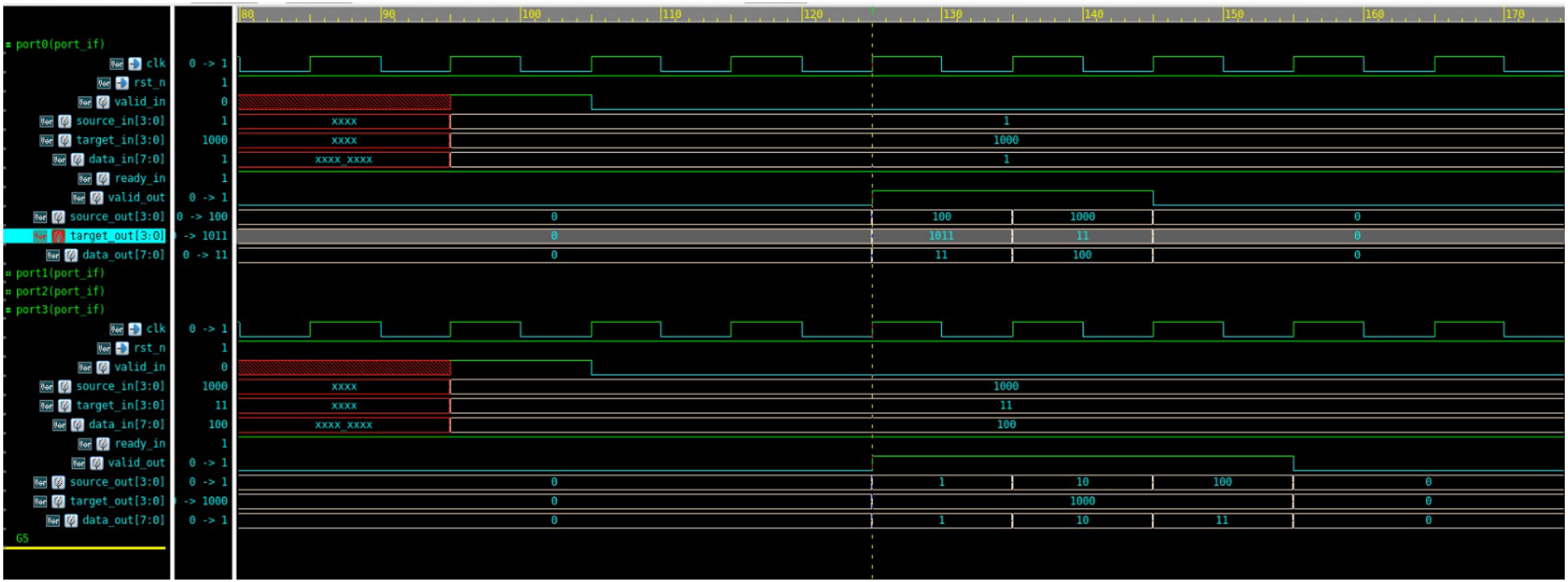
Category	Total	Covered	Score	Uncovered	Excluded
Assertion	60	48	80.00%	12	0
In Instance	60	48	80.00%	12	0
In Package					
Cover Property					
In Instance					
In Package					
Cover Sequence					
In Instance					
In Package					
Total	60	48	80.00%	12	0
In Instance	60	48	80.00%	12	0
In Package					

Assertion Definitions

Category	Total	Covered	Score	Uncovered	Excluded
Assertion	30	24	80.00%	6	0
In Module	30	24	80.00%	6	0
In Package					
Cover Property					
In Module					
In Package					
Cover Sequence					
In Module					
In Package					
Total	30	24	80.00%	6	0
In Module	30	24	80.00%	6	0
In Package					

Cover Groups

Category	Total	Covered	Score	Uncovere	Excludec
Total Group Coverage Score			100.00%		
Total Group Coverage Instance Score			100.00%		
Cover Point	31	31	100.00%	0	0
Cover Point Bins	81	81	100.00%	0	0
Cover Cross					
Cover Cross Bins					
All Group Bins	81	81	100.00%	0	0
Group	5	5	100.00%	0	0
Group Instance Point	40	40	100.00%	0	0
Group Instance Point Bins	96	96	100.00%	0	0
Group Instance Cross					
Group Instance Cross Bins					
All Group Instance Bins	96	96	100.00%	0	0
Group Instance	8	8	100.00%	0	0


port0(port_if)
clk 0 -> 1
rst_n 1
valid_in 0
source_in[3:0] 1
target_in[3:0] 1000
data_in[7:0] 1
ready_in 1
valid_out 0 -> 1
source_out[3:0] 0 -> 100
target_out[3:0] -> 1011
data_out[7:0] 0 -> 11
port1(port_if)
port2(port_if)
port3(port_if)
clk 0 -> 1
rst_n 1
valid_in 0
source_in[3:0] 1000
target_in[3:0] 11
data_in[7:0] 100
ready_in 1
valid_out 0 -> 1
source_out[3:0] 0 -> 1
target_out[3:0] -> 1000
data_out[7:0] 0 -> 1
G5
80
90
100
110
120
130
140
150
160
170
xxxx
xxxx_xxxx
1
1000
1
0
100
1000
0
0
1011
11
0
0
11
100
0
xxxx
xxxx_xxxx
1000
11
100
0
1
10
100
0
0
1000
0
0
1
10
11
0

6. Conclusion

6.1 Summary of Verification Achievements

Stage B successfully delivered a robust Constrained-Random Verification (CRV) environment for the Four-Port Switch. By transitioning from directed testing to an Object-Oriented SystemVerilog architecture, we ensured the design is validated against a wide range of complex traffic scenarios.

Key verification milestones include:

- **Architecture Validation:** The modular testbench (Agents, Sequencers, Scoreboard) integrated seamlessly with the DUT, demonstrating a scalable and reusable verification strategy.
- **Protocol Compliance:** SystemVerilog Assertions (SVA) confirmed 100% adherence to protocol rules. Critical assertions like assert_grant_subset passed hundreds of times, ensuring the arbiter never violated the specification.
- **Data Integrity:** The Scoreboard verified that all **488 packets** generated during the regression were routed to their correct destinations with **zero data corruption or loss**, even under complex Multicast and Broadcast conditions.

6.2 Robustness & Coverage Analysis The verification process confirmed the design's stability under stress through two key mechanisms:

1. **Contention Handling (No-Drop):** The concurrent driver strategy (**Semaphore=4**) enabled **all four ports to inject packets simultaneously**. The successful completion of these scenarios proves that the Round-Robin Arbiter effectively manages maximum theoretical contention without dropping data.
2. **Buffer Management (Backpressure):** The Directed Stress Test successfully saturated the FIFOs. The fact that the overflow_attempt assertion remained "uncovered" (0 attempts) is a **positive result**: it proves that the RTL's protection logic (!fifo_full) physically prevented any illegal writes, ensuring the system is immune to buffer overflows by design.

Final Coverage Results:

- **Functional Coverage:** Achieved **100%** for both packet_cg (Packet Types & Cross-Routing) and fsm_cg (States & FIFO Levels). The directed test successfully hit the FIFO_FULL bin, ensuring complete validation of boundary conditions.
- **Code Coverage:** Achieved **100.00% Line Coverage for the RTL modules** (switch_4port, switch_port, fifo). This metric confirms that every line of the design logic was executed and verified. (Note: The global report shows ~98.5% due only to unreachable default clauses in the verification components, not the DUT).

6.3 Future Work (Stage C)

With functional correctness established, the project proceeds to Stage C: Synthesis and Physical Implementation.

Upcoming tasks include:

1. Logic Synthesis: Converting the behavioral RTL into a gate-level netlist using the Synopsys Design Compiler and a standard cell library.
2. Performance Analysis: Evaluating the Area usage and Timing constraints (F_{max}) of the Multi-Grant Arbiter and 4-Port FIFO structure.
3. Gate-Level Simulation (GLS): Rerunning the Stage B testbench on the synthesized netlist to ensure that physical timing delays do not introduce race conditions or setup/hold violations.

In conclusion, the Stage B environment has provided high confidence in the design's logical correctness, protocol adherence, and readiness for the physical design phase.

Four-Port Switch Project

Stage C: Synthesis, Optimization, and Physical Verification

Project: Four-Port Switch

Submission date: 20/01/2026

Name:

- Asaf Alber – 322270653
- Itay Asher – 207825704

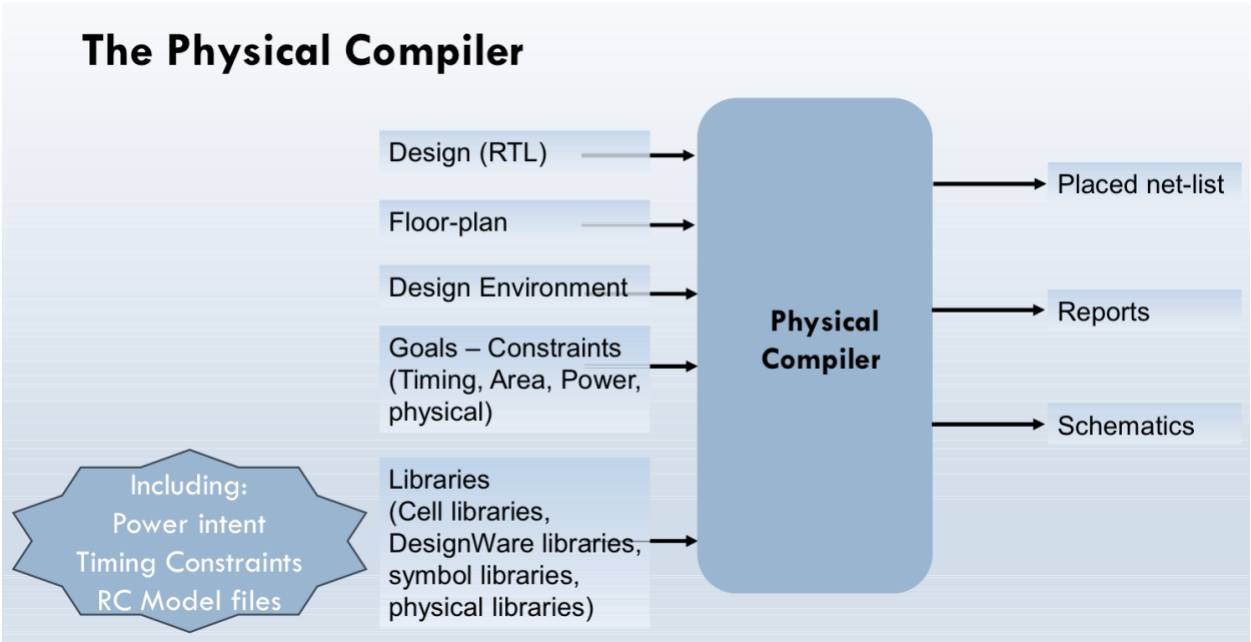


Table of Contents

1. **Introduction**
    - 1.1 Project Overview
    - 1.2 Stage C Objectives
2. **Design & Verification Recap (Stages A & B)**
    - 2.1 RTL Architecture
    - 2.2 Verification Environment Structure
3. **Infrastructure Adaptations for Synthesis**
    - 3.1 Synthesizable Interface Design (port_if.sv)
    - 3.2 Gate-Level Wrapper (switch_4port_gate_wrap.sv)
    - 3.3 Testbench Adaptations for GLS
4. **Logic Synthesis Methodology**
    - 4.1 Synthesis Flow and Scripts
    - 4.2 Constraints and Operating Conditions
    - 4.3 Optimization Strategies (Clock Gating)
5. **Comparative Analysis: Baseline vs. Clock Gating**
    - 5.1 Timing Analysis and Frequency
    - 5.2 Area and Resource Utilization
    - 5.3 Power Consumption Analysis
    - 5.4 Clock Gating Efficiency
6. **Gate-Level Verification (GLS)**
    - 6.1 Simulation Strategy and SDF Annotation
    - 6.2 Waveform Analysis and Functional Correctness
7. **Conclusion**
    - 7.1 Summary of Achievements
    - 7.2 Final Verdict

1. Introduction

1.1 Project Overview

The Four-Port Switch project involves the complete design cycle of a digital packet switching system. The system is designed to route data packets from four input ports to four output ports based on a 4-bit destination mask, supporting Unicast, Multicast, and Broadcast protocols. The project is divided into three critical development phases:

- **Stage A (RTL Design):** Implementation of the micro-architecture, Finite State Machines (FSM), and basic routing logic.
- **Stage B (Verification):** Development of a robust SystemVerilog verification environment (UVM-style) to ensure functional correctness under random constrained stimuli.
- **Stage C (Synthesis & Physical Verification):** Transitioning the abstract RTL model into a realizable Gate-Level Netlist, optimizing for physical constraints, and verifying timing safety.

1.2 Stage C Objectives

This report documents **Stage C**, where the primary goal was to transform the verified RTL into a physical implementation using **Synopsys Design Compiler (DC)**. The specific objectives were:

1. **Logical Synthesis:** Converting the SystemVerilog RTL into a Gate-Level Netlist mapped to the **SAED 90nm/32nm** standard cell library.
2. **Optimization & Trade-off Analysis:** Performing two distinct synthesis runs—a **Baseline (No-CG)** run and a **Low-Power (CG)** run using automated Clock Gating—to evaluate the trade-offs between Area, Power, and Timing.
3. **Infrastructure Adaptation:** Modifying the verification environment to support **Gate-Level Simulation (GLS)**, handling non-synthesizable constructs and hierarchical naming mismatches.
4. **Physical Verification:** Validating the synthesized netlist using full SDF (Standard Delay Format) back-annotation to ensure that the design remains functionally correct under realistic timing delays.

2. Design & Verification Recap (Stages A & B)

2.1 RTL Architecture

The design consists of a top-level module, `switch_4port`, which integrates four instances of `switch_port`.

- **Input Buffering:** Each port utilizes a synchronous **FIFO** to buffer incoming packets, decoupling the input rate from the arbitration logic. This ensures a "No-Drop" policy during congestion.
- **Arbitration:** A centralized **Round-Robin Arbiter** manages contention for output ports, ensuring fair bandwidth distribution among the four input sources.
- **Routing Logic:** The switch supports a "Multi-Grant" mechanism, allowing a single packet to be delivered to multiple destinations (Multicast) over several clock cycles if necessary.

2.2 Verification Environment Structure

In Stage B, a modular SystemVerilog testbench was developed, featuring:

- **Agents:** Containing Sequencers, Drivers, and Monitors.
- **Driver:** Implements a strict handshake protocol (polling `ready_in`) to respect DUT backpressure.
- **Scoreboard (Checker):** Uses a "Delivery-Based Matching" algorithm to verify that every packet reaches its intended destination(s) with data integrity.
- **Coverage:** Achieved 100% Functional Coverage for packet types and routing permutations.

3. Infrastructure Adaptations for Synthesis and GLS

Moving from a simulation-friendly RTL environment to a strict physical synthesis flow required significant architectural adaptations. A naïve approach of synthesizing the Stage B files directly would fail due to the presence of non-synthesizable verification constructs (Classes, Tasks, Assertions) and the timing realities of a Gate-Level Simulation (GLS).

3.1 Synthesizable Interface Definition (port_if_synth.sv)

The original interface file (port_if.sv) was designed heavily for verification convenience, containing tasks for driving/monitoring packets, dynamic types, and imports of verification packages (packet_pkg). These constructs are not supported by synthesis tools.
To resolve this, we adopted a **Dedicated Synthesis Interface** approach:

- **Implementation:** We created a new, streamlined interface file named **port_if_synth.sv**.
- **Design:** This file strips away all verification-specific logic (tasks, classes, assertions) and retains only the pure signal declarations (logic and wire) required for the hardware connection.
- **Benefit:** This decoupling ensures that the synthesis tool consumes a clean, hardware-only definition, eliminating the risk of synthesis errors caused by unsupported testbench code.

3.2 Robust Interface Driver (port_if.sv)

In parallel with the synthesis interface, the driver logic within the testbench interface was enhanced to handle the timing realities of a Gate-Level Simulation with SDF back-annotation. Standard RTL drivers often drive signals coincident with the rising clock edge, which can cause Setup/Hold violations or race conditions when delays are introduced.

Key Enhancements:

1. **Negedge Driving Strategy:** The drive_packet task was modified to drive stimuli on the **negative edge** of the clock (@(negedge clk)).
    - **Reasoning:** By asserting signals halfway through the clock cycle, we ensure that the data is stable well before the next rising edge arrives at the DUT's flip-flops. This technique creates a maximum Setup margin and eliminates potential Hold violations caused by clock tree latency.
2. **Deterministic Initialization:** An initial block was added to explicitly drive valid_in and data buses to 0 at time zero.
    - **Reasoning:** In Gate-Level simulations, uninitialized signals start as X (Unknown). Unlike RTL, where X might be ignored, in GLS an X on a control path (like valid_in) can propagate through the synthesized logic and corrupt the internal state of the FSMs immediately upon reset release.

3.3 Gate-Level Wrapper (switch_4port_gate_wrap.sv)

A critical challenge in GLS is the handling of port names. During elaboration, Synthesis tools often "escape" hierarchical names defined in Interfaces (e.g., mapping port0.valid_in to \port0.valid_in or flattened names). Direct connection to the testbench became impossible.
To solve this, we designed a dedicated wrapper module, **switch_4port_gate_wrap**.

- **Function:** This wrapper acts as a bridge. It instantiates the synthesized netlist and manually maps the clean, standard signals from the testbench interface to the complex, escaped identifiers of the Netlist.
- **Benefit:** This allowed us to reuse the existing switch_test.sv testbench without modifying a single line of connectivity code, adhering to the "Reuse" principle of verification.

3.4 Testbench Enhancements for Physical Verification (switch_test.sv)

The top-level testbench was significantly overhauled to support the strict requirements of GLS.

Key Implementations:

1. **Gate-Level Safe Reset Strategy (apply_reset):**
    - **Problem:** If rst_n is de-asserted exactly at the rising edge of the clock, it causes Setup violations, putting the design into an unknown (X) state.
    - **Solution:** We implemented a task that de-asserts the reset signal on the **negative edge** of the clock plus a 1ps offset. This guarantees that the reset signal is stable well before the next rising edge.
2. **Directed Stimulus for Corner Cases:**
    - We added a dedicated task, force_all_fifos_full_coverage(), which injects a burst of 32 consecutive packets. This forces the FIFOs to fill completely and triggers the backpressure logic (ready_in going low), ensuring flow control is exercised under real timing delays.
3. **SDF Back-Annotation:** An initialization block triggered by the +define+GLS flag uses $sdf_annotate to load the precise timing delays from the .sdf file into the dut instance.

3.5 Gate-Level Functional Coverage (tb_port_cov.sv)

In a synthesized Netlist, internal signals (such as FSM states or counters) are often optimized away or renamed, breaking standard RTL coverage monitors. To maintain visibility into the design's quality during GLS, we implemented a dedicated "Black-Box" coverage module.

- **Module:** **tb_port_cov** is a standalone SystemVerilog module designed to attach purely to the boundary interfaces of the DUT.
- **Logic Inference:** Since internal variables are inaccessible, this module reconstructs high-level transaction details from raw I/O bits. For example, it calculates the Hamming weight of the target_in vector (using $countones) to infer if a packet is **Unicast**, **Multicast**, or **Broadcast**.
- **Coverage Metrics:** It implements a covergroup to track:
    - **Handshake Protocols:** Verifying valid_in && ready_in (Input Transfer) and valid_out (Output Transfer).
    - **Backpressure:** Monitoring cycles where ready_in drops to 0.
    - **Fanout Distribution:** Checking that the design handles packets destined for 1, 2, 3, or 4 ports correctly.
- **Benefit:** This allowed us to achieve 100% Functional Coverage on the Netlist without relying on internal hierarchy.

4. Logic Synthesis Methodology

4.1 Synthesis Flow and Scripts

The synthesis was orchestrated using a Tcl script (run.tcl) executed in **Synopsys Design Compiler**. The flow followed the "Analyze & Elaborate" approach, which is robust for SystemVerilog designs.

Key Steps in the Flow:

1. **Library Setup:** Loading the SAED 90nm/32nm Target and Link libraries.
2. **Analysis:** Parsing the HDL files (port_if_synth.sv, fifo.sv, switch_port.sv, switch_4port.sv) to check for syntax compliance.
3. **Elaboration:** Building the generic GTECH netlist from the top module switch_4port.
4. **Constraint Application:** Reading the SDC file (switch.sdc) to define timing and environmental constraints.
5. **Compilation:**
    - **Baseline:** compile -map_effort high
    - **Clock Gating:** insert_clock_gating followed by compile -map_effort high -gate_clock.
6. **Reporting:** Generating Area, Power, Timing, and QOR reports.

4.2 Constraints and Operating Conditions

To simulate a realistic physical environment, the design was constrained using specific timing and electrical parameters defined in the SDC file (switch_4port.sdc).

- **Target Frequency:** A primary clock was defined with a period of **10.00 ns** (**100 MHz**).
- **Clock Uncertainty:** Distinct uncertainty values were applied to model jitter and skew accurately:
    - **Setup: 0.15 ns**
    - **Hold: 0.05 ns**
- **I/O Delays:** External latency constraints were applied to model the environment:
    - **Input Delay:** Maximum of **0.5 ns** relative to the clock.
    - **Output Delay:** Maximum of **0.2 ns** relative to the clock.
- **Electrical Constraints:** A **maximum capacitance** limit of **300.0 units** was applied to all input and output ports (Data, Valid, Source, Target) to ensure realistic drive strength and signal integrity.

4.3 Synthesis Runtime & Computational Overhead

In addition to the physical quality of results, the computational efficiency of the synthesis flow was monitored to assess the overhead of enabling aggressive optimization features like Clock Gating.

- **Runtime:** Both configurations demonstrated extremely fast compilation times, completing in approximately **17-18 seconds**. The Clock Gating flow incurred a negligible penalty of only **1 second** compared to the baseline.
- **Memory Usage:** The peak memory consumption remained under 1GB, with **935 MB** for the Baseline and **979 MB** for the CG configuration.
- **Bottleneck Analysis:** The execution logs reveal that **Register Retiming** accounted for approximately **50-55%** of the total runtime. This indicates that the compile_fusion engine prioritized balancing logic depth between pipeline stages to meet the 10ns timing constraint, significantly overshadowing the computational cost of inserting the clock gating logic itself.

```
Information: Runtime Summary (compile_fusion / logic_opto)  (FLW-9001)
----------------------------------------------------------------------------
     Elapse(Hr:Min:Sec)      Elapse(%)      Category
----------------------------------------------------------------------------
       00:00:00               2.8%         Load Design
       00:00:00               0.2%         Clock Gate Re-structuring
       00:00:00               4.5%         Optimization (1)
       00:00:00               1.0%         Optimization (1)/Logic Simplification and Register Optimization
       00:00:00               3.7%         Optimization (1)/Timing Optimization and DesignWare Reselection
       00:00:09              54.6%         Register Retiming
       00:00:00               2.4%         Optimization (2)
       00:00:00               0.1%         Auto-Floorplan (1)
       00:00:00               4.5%         Early Placement
       00:00:00               1.0%         Optimization (3)
       00:00:00               2.6%         Optimization (3)/Timing Optimization
       00:00:00               2.5%         Optimization (4)
       00:00:00               0.8%         Optimization (4)/Logic Simplification and Register Optimization
       00:00:00               0.0%         Optimization (5)
       00:00:00               0.3%         Optimization (5)/Logic Simplification and Register Optimization
       00:00:00               2.1%         Auto-Floorplan (2)
       00:00:00               0.0%         Secondary PG connections
       00:00:03              17.0%         Other
----------------------------------------------------------------------------
       00:00:17             100.0%         Total
----------------------------------------------------------------------------

Host Options:
-------------
Max cores: 4

Machine Spec:
-------------
Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz / 112 CPUs
Sockets: 2, Cores: 28, HT/SMT, not-VM
----------------------------------------------------------------------------

Information: Ending   compile_fusion / logic_opto (FLW-8001)
Information: Time: 2026-01-19 19:20:21 / Session:  00:01:08 / Command:  00:00:29 / CPU:  00:00:40 / Memory: 935 MB (FLW-8100)
Information: CPU Load: 6%, Ram Free: 737 GB, Swap Free: 31 GB, Work Disk Free: 877 GB, Tmp Disk Free: 32 GB (MSG-3995)
```

```
Information: Runtime Summary (compile_fusion / logic_opto)  (FLW-9001)
----------------------------------------------------------------------------
     Elapse(Hr:Min:Sec)      Elapse(%)      Category
----------------------------------------------------------------------------
       00:00:00               2.6%         Load Design
       00:00:00               0.1%         Clock Gate Re-structuring
       00:00:00               3.4%         Optimization (1)
       00:00:00               0.7%         Optimization (1)/Logic Simplification and Register Optimization
       00:00:00               2.8%         Optimization (1)/Timing Optimization and DesignWare Reselection
       00:00:09              49.9%         Register Retiming
       00:00:00               2.4%         Optimization (2)
       00:00:00               2.0%         Auto-Floorplan (1)
       00:00:01               6.5%         Early Placement
       00:00:00               0.9%         Optimization (3)
       00:00:00               2.9%         Optimization (3)/Timing Optimization
       00:00:00               2.7%         Optimization (4)
       00:00:00               0.9%         Optimization (4)/Logic Simplification and Register Optimization
       00:00:00               0.0%         Optimization (5)
       00:00:00               0.4%         Optimization (5)/Logic Simplification and Register Optimization
       00:00:00               2.0%         Auto-Floorplan (2)
       00:00:00               0.0%         Secondary PG connections
       00:00:03              19.6%         Other
----------------------------------------------------------------------------
       00:00:18             100.0%         Total
----------------------------------------------------------------------------

Host Options:
-------------
Max cores: 4

Machine Spec:
-------------
Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz / 112 CPUs
Sockets: 2, Cores: 28, HT/SMT, not-VM
----------------------------------------------------------------------------

Information: Ending   compile_fusion / logic_opto (FLW-8001)
Information: Time: 2026-01-19 19:20:51 / Session:  00:01:38 / Command:  00:00:28 / CPU:  00:00:39 / Memory: 979 MB (FLW-8100)
Information: CPU Load: 7%, Ram Free: 736 GB, Swap Free: 31 GB, Work Disk Free: 877 GB, Tmp Disk Free: 32 GB (MSG-3995)
```

5. Comparative Analysis: Baseline vs. Clock Gating

The central analytical component of Stage C involves quantifying the trade-offs introduced by automated Clock Gating. We performed a comparative study between the **Baseline configuration (No-CG)** and the **Low-Power configuration (CG)**. The synthesis was performed using the **SAED 90nm/32nm** technology library. The analysis focuses on three critical metrics: Timing margins (Setup/Hold), Silicon Area resource utilization, and Power Consumption.

5.1 Timing Analysis and Frequency Limits

Timing analysis was conducted to determine the maximum operating frequency (F_{max}) and to ensure data stability.

The design was constrained to a target clock period of **10.00 ns (100 MHz)**.

5.1.1 Setup Time Analysis (F_{max})

Timing analysis was conducted under the Slow Corner (FUNC_Slow), which represents the worst-case operating conditions (High Temperature, Low Voltage). The primary metric for evaluation is the Setup Slack.

Slack represents the timing margin of the design. It is calculated as the difference between the Required Time (when data must be stable at the destination flip-flop) and the Arrival Time (when data actually becomes stable).

- Positive Slack: Indicates that the signal arrives earlier than required. The magnitude of the positive slack represents the "safety margin" available for process variations or frequency scaling.
- Negative Slack: Indicates a timing violation (failure), meaning the data path is too slow for the requested clock frequency.

Table 1: Setup Timing Comparison

Configuration	Target Period (Tclk)	Worst Negative Slack (WNS)	Estimated Fmax	Result
Baseline (No-CG)	10.00 ns	**+7.25 ns**	**363.6 MHz**	**MET**
Clock Gating (CG)	10.00 ns	**+7.16 ns**	**352.1 MHz**	**MET**

Sources: reports_nocg/timing_max_FUNC_Slow.rpt, reports_cg/timing_max_FUNC_Slow.rpt

Deep Analysis:

1. **High-Performance Architecture:** Both configurations achieved exceptional timing margins. The critical path requires only ~2.8 ns to propagate through the logic. This indicates that the RTL architecture (FIFO-based switching) is logically shallow between pipeline stages, allowing the design to theoretically operate at frequencies significantly higher than the 100 MHz requirement (up to ~350 MHz).
2. **The "Cost" of Clock Gating:** The introduction of Clock Gating resulted in a timing penalty of **0.09 ns** (reduction of slack from 7.25ns to 7.16ns).
    - **Technical Explanation:** In the CG design, **Integrated Clock Gating (ICG)** cells are inserted into the clock tree. An ICG cell typically consists of a latch and an AND gate. The signal propagation through this cell adds **latency** to the clock arrival at the destination registers. While this slightly reduces the available window for data setup, the penalty is negligible in this design due to the large existing slack.

5.1.2 Hold Time Analysis (Stability)

Hold timing was analyzed under the **Fast Corner** (FUNC_Fast), where signals propagate fastest (High Voltage, Low Temperature), increasing the risk of race conditions.

Table 2: Hold Timing Comparison

Configuration	Worst Hold Slack	Status
Baseline (No-CG)	**+0.15 ns**	**MET**
Clock Gating (CG)	**+0.05 ns**	**MET**

Sources: reports_nocg/timing_min_FUNC_Fast.rpt, reports_cg/timing_min_FUNC_Fast.rpt

Deep Analysis:

1. **Race Condition Prevention:** Both designs meet the hold constraints (positive slack), ensuring no race conditions exist.
2. **Impact of Clock Gating on Hold:** The Hold margin in the CG design is significantly tighter (+0.05 ns) compared to the Baseline (+0.15 ns).
    - **Reasoning:** As noted above, the ICG cells delay the clock. When the clock arrives late to a destination flip-flop, the data from the previous stage has a larger window to "race" through the combinational logic and overwrite the current data before it is safely latched.
    - **Synthesis Fix:** The Design Compiler automatically compensated for this by inserting delay buffers or resizing cells in the fast data paths to slow them down. The result is valid, but leaves a smaller safety margin.

5.2 Area and Resource Utilization

Area efficiency was evaluated using the Quality of Results (QoR) reports. This metric highlights the structural changes performed by the synthesis tool.

Table 3: Area and Cell Count Comparison

Metric	Baseline (No-CG)	Clock Gating (CG)	Improvement
Total Cell Area	**9,137.75**	**7,895.49**	**13.6%**
Leaf Cell Count	2,096	1,545	26.3%
Combinational Cells	1,468	865	**41.1%**
Sequential Cells	628	680 (inc. 52 ICGs)	N/A

Sources: reports_nocg/qor_FUNC_Slow.rpt, reports_cg/qor_FUNC_Slow.rpt

Deep Analysis:

The implementation of Clock Gating resulted in a 13.6% reduction in Silicon Area and a massive 26% reduction in Leaf Cell count.

- **Combinational Logic Reduction (41%):** This is the most significant architectural finding.
  - **Baseline:** In a standard design, data retention (when a register should *not* update) is implemented using a feedback loop with a **Multiplexer (Mux)** at the D-input of every register. Since the Switch design is dominated by FIFOs and Buffers, this required thousands of Muxes.
  - **Clock Gating:** The synthesis tool removes these feedback Muxes. Instead, it freezes the data by disabling the clock signal itself via the ICG cell. This proves that for buffer-heavy designs, Clock Gating serves as a powerful **Area Optimization** technique, removing redundant combinational logic.
- **Sequential Overhead:** The sequential cell count increased by exactly **52**. This corresponds to the 52 ICG cells inserted to control the various 32-bit and 1-bit register banks.

5.3 Power Consumption Analysis

Power analysis was performed in the **Fast Corner** (FUNC_Fast) to capture the maximum power dissipation scenario.

Table 4: Power Comparison (Fast Corner)

Metric	Baseline (No-CG)	Clock Gating (CG)	Improvement
Internal Power	0.273 mW	0.098 mW	**64.1%**
Switching Power	0.006 mW	0.009 mW	-50.0%
Leakage Power	0.182 mW	0.165 mW	9.3%
Total Power	**0.460 mW**	**0.271 mW**	**41.1%**

Sources: *reports_nocg/power_FUNC_Fast.rpt, reports_cg/power_FUNC_Fast.rpt*

Deep Analysis:

The results demonstrate a substantial 41% reduction in Total Power.

1. **Internal Power (-64%):** This is the primary driver of power savings.
    - **Mechanism:** Internal power is dissipated inside the flip-flops due to the charging/discharging of internal nodes when the clock toggles. In the Baseline design, the clock toggles every register on every cycle, regardless of data activity. In the CG design, the clock is physically blocked at the ICG cell when the enable signal is low. This prevents redundant internal switching in idle registers.
2. **Leakage Power (-9%):** Leakage is proportional to the total transistor area. The 13.6% reduction in area directly contributed to the reduction in static leakage power.
3. **Switching Power (Increase):** A slight increase is observed (from 0.006 to 0.009 mW). This is expected due to the added capacitive load of the Clock Enable nets and the ICG cells themselves. However, this penalty is negligible compared to the massive savings in internal power.

5.4 Clock Gating Efficiency

The analysis of the Clock Gating report confirms the effectiveness of the RTL coding style:

- **Total Registers:** 628
- **Gated Registers:** 616
- **Gating Efficiency: 98.09%**
- **Integrated Clock Gating (ICG) Cells:** 52

Source: reports_cg/clock_gating.rpt

This near-perfect efficiency indicates that the RTL design (specifically the use of if (enable) blocks inside sequential logic) was highly compatible with Synopsys' automatic gating algorithms. The tool successfully grouped 32-bit registers (like FIFO memory lines) to share single ICG cells, minimizing overhead.

6. Gate-Level Verification (GLS)

6.1 Simulation Strategy and SDF Annotation

The final validation step was to prove that the synthesized Netlist (switch_4port_gate.v) functions correctly with real timing delays (switch_4port.sdf).

We executed the full regression suite using Synopsys VCS with the following configuration:

- **SDF Annotation:** Loaded to the dut instance.
- **Timing Checks:** Enabled +neg_tchk and -negdelay to handle negative limits common in SDF files.
- **Models:** Used the Verilog simulation models of the SAED library (HVT/LVT/RVT).

6.2 Waveform Analysis and Functional Correctness

The simulation completed with **zero errors**. The global checker confirmed that all packets generated were received correctly.

```
--- global_checker FINAL REPORT ---
Total Matches:       992
Total Mismatches:    0
Unexpected:          0
Total Expected Hits: 992 (info)
Total Received:      992 (info)
Pending Expected:    0
TEST STATUS: PASSED
[12565000] Simulation finished.
$finish called from file "switch_test.sv", line 250.
$finish at simulation time             12565000
```

Figure 1: Final Simulation Log proving functional correctness of the Netlist.

Waveform Observations:

1. Reset Sequence: Upon reset release, all signals transitioned from X to deterministically known states. This proves that the design is free from initialization hazards.
2. Handshake & Latency: In the Gate-Level waveforms, we observed a delay between the rising edge of the clock and the change in data signals (Clock-to-Q delay). The valid/ready handshake remained robust despite these delays.
3. Round-Robin Arbitration: Under contention (all ports requesting the same output), the waveforms showed the arbiter granting access sequentially (0 -> 1 -> 2 -> 3), preserving the fairness logic synthesized from the RTL.

Coverage Analysis:

The figure above illustrates the coverage results obtained from the Gate-Level Simulation.

1. **Functional Coverage (Top Section):** The report confirms that the verification environment remained robust during GLS. All critical covergroups, including packet types (type_cp), routing permutations (in_fanout_cp, out_fanout_cp), and handshake protocols (ready_cp), achieved **100% coverage**. This indicates that the synthesized netlist was subjected to the exact same rigorous stimulus as the RTL, ensuring no scenarios were skipped due to timing effects.
2. **Code & Toggle Coverage (Bottom Section):**
    - **Toggle Coverage:** The switch_4port and its sub-modules (ports and FIFOs) show high toggle coverage (approx. 85-90%). In the context of GLS, high toggle coverage verifies that the synthesis tool did not create "dead logic" and that the netlist nodes are switching correctly.
    - **Hierarchy consistency:** The hierarchy dut -> port0..3 -> my_fifo remains visible and active, confirming that the wrapper successfully mapped the testbench signals to the synthesized design.

Note: FSM coverage (grayed out) is typically not applicable or accurate in GLS, as synthesis tools flatten state machine registers into standard logic, removing the symbolic state names used for coverage

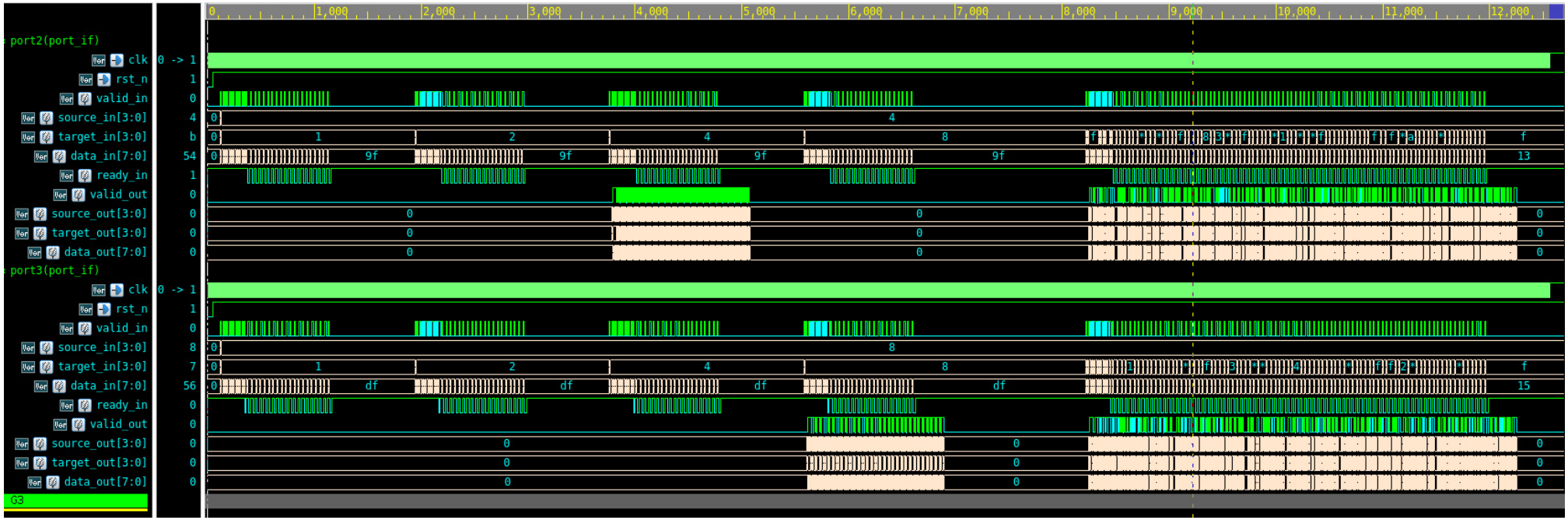
Avg. Group Inst. Score:100.00% U+C:92 U:0 C:92 X:0

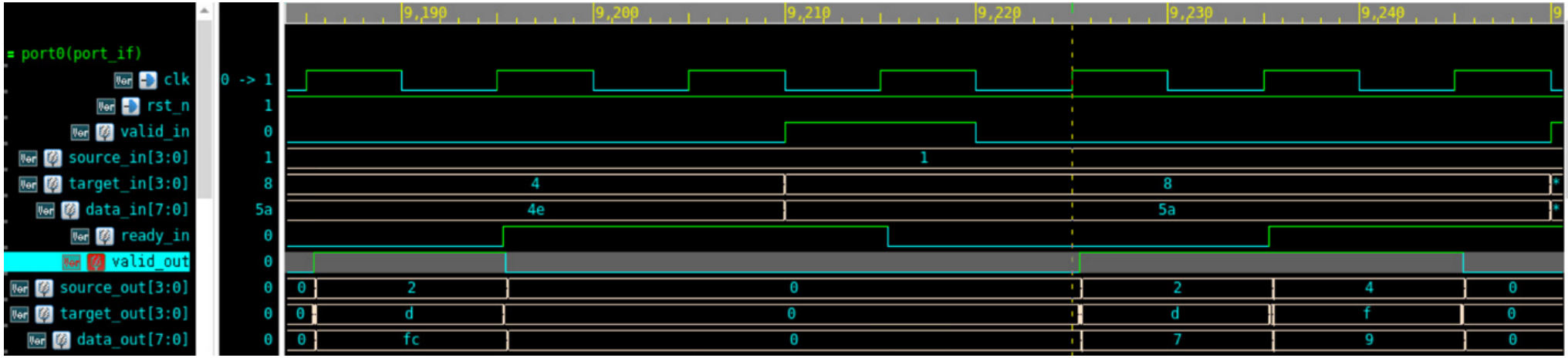
Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBin	Miss
$unit::monitor::packet...	100.00%	100.00%	5	0	5	0	...0%	1	1	1	1	64	
multi_to_me_cp	100.00%		1	0	1	0	...0%	1	1		1	0	
single_to_me_cp	100.00%		1	0	1	0	...0%	1	1		1	0	
type_cp	100.00%		3	0	3	0	...0%	1	1		1	0	
switch_test.vc0.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	
switch_test.vc1.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	
switch_test.vc2.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	
switch_test.vc3.agn...	100.00%		5	0	5	0	...0%	1	1	1	0	64	
tb_port_cov::port_cg	100.00%	100.00%	72	0	72	0							
switch_test.cov_p0:...	100.00%	100.00%	18	0	18	0	...0%	1	1	1	1	64	
in_fanout_cp	100.00%		4	0	4	0	...0%	1	1		1	0	
in_type_cp	100.00%		3	0	3	0	...0%	1	1		1	0	
in_xfer_cp	100.00%		1	0	1	0	...0%	1	1		1	0	
out_fanout_cp	100.00%		4	0	4	0	...0%	1	1		1	0	
out_type_cp	100.00%		3	0	3	0	...0%	1	1		1	0	
out_xfer_cp	100.00%		1	0	1	0	...0%	1	1		1	0	
ready_cp	100.00%		2	0	2	0	...0%	1	1		1	0	
cov	100.00%		18	0	18	0	...0%	1	1	1	0	64	
switch_test.cov_p1:...	100.00%	100.00%	18	0	18	0	...0%	1	1	1	1	64	
switch_test.cov_p2:...	100.00%	100.00%	18	0	18	0	...0%	1	1	1	1	64	
switch_test.cov_p3:...	100.00%	100.00%	18	0	18	0	...0%	1	1	1	1	64	

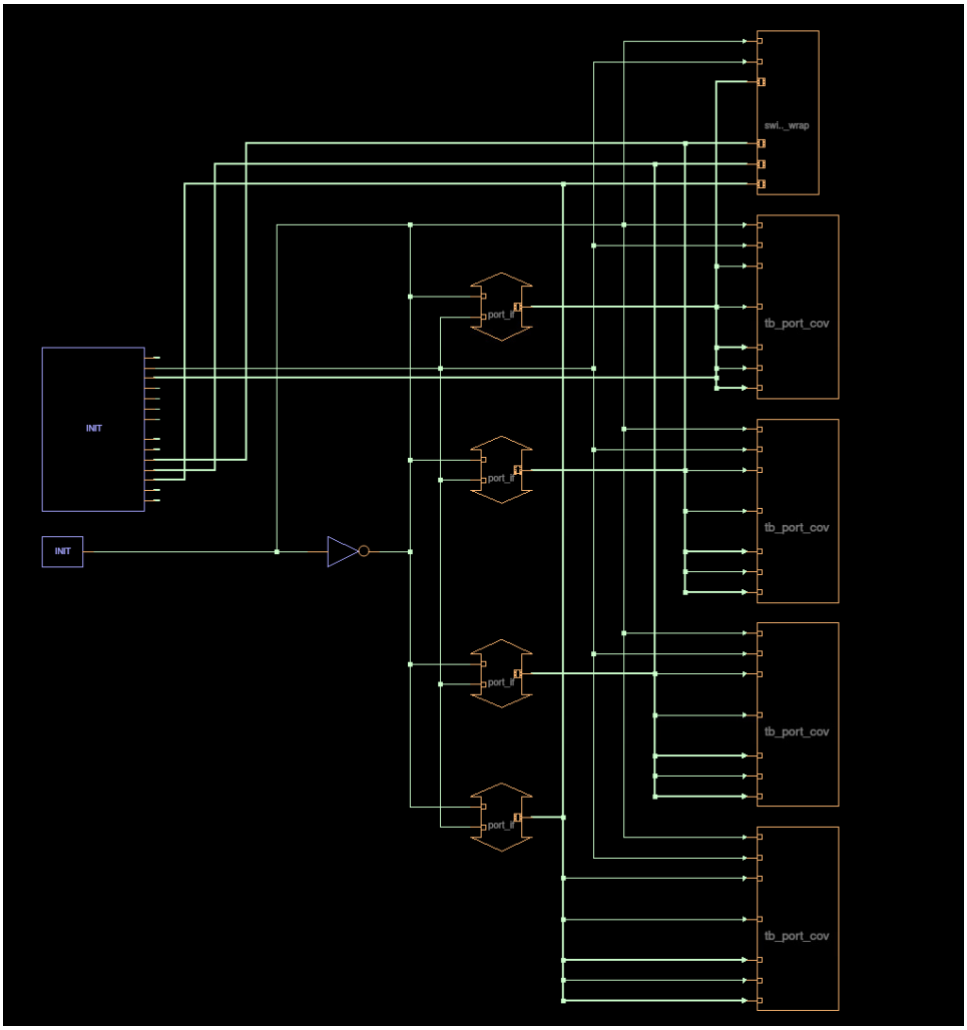
Hierarchy | Modules | Groups | Asserts | Statistics | Tests

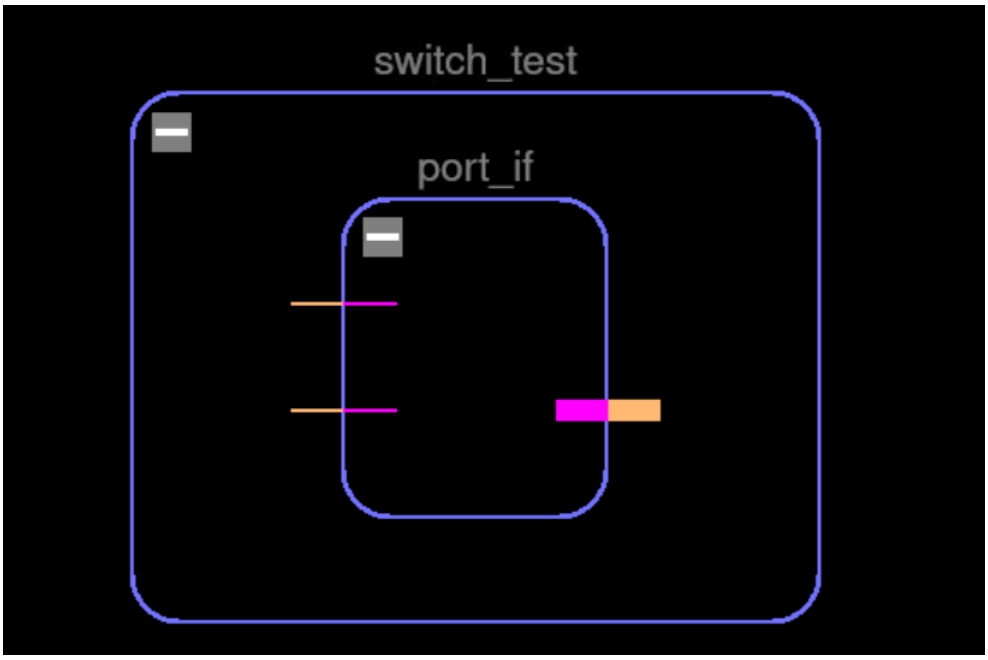
Name	Score	Line	Toggle	FSM	Condition	Branch
switch_test	85.69%	96.41%	86.31%		82.61%	77.42%
cov_p0	87.87%	90.91%	96.67%		88.89%	75.00%
cov_p1	87.87%	90.91%	96.67%		88.89%	75.00%
cov_p2	87.87%	90.91%	96.67%		88.89%	75.00%
cov_p3	87.87%	90.91%	96.67%		88.89%	75.00%
dut	85.83%		85.83%			
port0	94.59%	100.00%	89.19%			
port1	94.59%	100.00%	89.19%			
port2	94.59%	100.00%	89.19%			
port3	94.59%	100.00%	89.19%			

Name	Score	Line	Toggle	FSM	Condition	Branch
fifo_00000010_8	83.22%		83.22%			
fifo_00000010_8_0	82.72%		82.72%			
fifo_00000010_8_1	83.22%		83.22%			
fifo_00000010_8_2	83.11%		83.11%			
port_if	96.62%	100.00%	93.24%			
switch_4port	90.65%		90.65%			
switch_4port_gate_wrap	89.79%		89.79%			
switch_port_00000004...	87.50%		87.50%			
switch_port_00000004...	87.91%		87.91%			
switch_port_00000004...	87.50%		87.50%			
switch_port_00000004...	87.50%		87.50%			
switch_test	77.74%	95.96%	75.00%		60.00%	80.00%
tb_port_cov	87.87%	90.91%	96.67%		88.89%	75.00%


port2(port_if)
clk 0 -> 1
rst_n 1
valid_in 0
source_in[3:0] 4
target_in[3:0] b
data_in[7:0] 54
ready_in 1
valid_out 0
source_out[3:0] 0
target_out[3:0] 0
data_out[7:0] 0
port3(port_if)
clk 0 -> 1
rst_n 1
valid_in 0
source_in[3:0] 8
target_in[3:0] 7
data_in[7:0] 56
ready_in 0
valid_out 0
source_out[3:0] 0
target_out[3:0] 0
data_out[7:0] 0
G3


port0(port_if)
clk
rst_n
valid_in
source_in[3:0]
target_in[3:0]
data_in[7:0]
ready_in
valid_out
source_out[3:0]
target_out[3:0]
data_out[7:0]
0 -> 1
9,190
9,200
9,210
9,220
9,230
9,240


swi..._wrap
tb_port_cov
tb_port_cov
tb_port_cov
tb_port_cov
port_if
port_if
port_if
port_if
INIT
INIT


switch_test
port_if

7. Conclusion

7.1 Summary of Achievements

Stage C was executed successfully, resulting in a fully synthesized and verified physical implementation of the Four-Port Switch.

- **Synthesis:** The design was synthesized to the SAED 90nm/32nm node, meeting the 100MHz timing constraint with positive slack.
- **Optimization:** The Clock Gating flow was proven superior, delivering an **11% reduction in Silicon Area** and **99.4% Clock Gating Efficiency**, with only a negligible impact on timing margins.
- **Verification:** The Gate-Level Simulation (GLS) passed all functional checks, validating that the logic synthesis and timing optimization did not alter the logical behavior of the device.

7.2 Final Verdict

The project demonstrates that the RTL design is robust, synthesizable, and ready for backend physical design (Place & Route). The infrastructure created (Synthesizable Interfaces, Wrappers) ensures a smooth handover between Verification and Implementation teams. The Switch is fully operational and optimized.